
Dalek：一台构造性 Agent 机器

以构造实现自维护、自进化、自复制与自组织

Wanpeng Xie
wanpeng.xie@gmail.com

Abstract

本文提出 Dalek：一台为 agent 设计的封闭机器。它可以在任何满足一般宿主契约的底座上实现自维护、自进化、自复制与自组织。机器由 actor、message 与 channel 三种原语构成；宿主边界、构造语言、合法转移与规则遗传四项义务，为它的边界、身份与封闭性提供结构基础。

冯诺依曼 1948 年的自复制自动机提供了一个可遗传的构造核：一份自我描述，以及构造器、复印器和控制器。Dalek 将这一构造核与四项义务结合，并针对文本与消息构成的 agent 底座重新导出其介质，加入边界、身份、历史与增长的显式结构。大语言模型与编译器占据载荷位，共同构成一个通用能力制造器。新能力由它写出、编译、装入描述并由后代继承；机器自身的器官乃至运行时也通过同一路径产生，使遗传与进化闭合于机器之内。

关键词：agent 系统；自复制自动机；自修改系统；构造性定义；软件架构

目录

1 引言	2
2 构造模型与底座	3
3 机器	5
4 运行中的机器：从一个任务到第三代	14
5 讨论	22
6 相关工作	24
7 结论与未来工作	26
记号	27
A 转移表与 ABI	28
B 证据索引	30

1 引言

1.1 从清单到机器

Agent 通常被实现为围绕大语言模型组织的一组部件：模型负责生成，记忆、工具、工作流、子 agent、网关与验证器承担其余工作。面对更困难的任务，一种办法是不断加长这张清单，为每一种新需求配置一种新部件。清单越长，装配、配置与维护的代价也越高；能力以系统复杂度为代价购得。

另一种办法是让模型在运行时生成和组装部件，动态插件与 code mode 是其中的代表 [Shi et al., 2026, DeepSeek-AI, 2026]。生成取代了预装，因此新能力不必事先枚举。更深的问题发生在生成之后：agent 写出的代码也许立刻可调用，但它是否已经成为 agent 的一部分，能否跨过重启，是否会被副本继承，又是否获准装入，仍由文件系统、加载器、镜像构建器、部署脚本或操作者决定。同一份代码究竟只是一次任务的产物，还是系统自身的一次修改，取决于 agent 之外的安排。生成新结构的能力已经进入 agent；裁定什么构成 agent 的权力却没有。

失控的复杂度与外移的构成权有同一个根源：**agent 机器缺少一个构造性定义。**

当 agent 只服务于预定任务，并始终由人持续维护时，这一缺失并不重要：工具不必定义自己。但当任务无法事先枚举、运行不以人在场为前提，并且改进对象最终包含系统自身的结构时，要求的性质就发生了变化。旧主张是存在性的：系统完成过一个任务、无人值守运行过若干小时，或者曾经重写过一次自己的 scaffold；一次演示足以证明它。新主张是不变性的：无论系统运行多久、收到什么任务、结构改变多少次，§1.2 的性质始终成立。一次演示只能说明某件事发生过一次。要说它始终为真，就需要结构保证；而在陈述这种保证之前，必须先定义承受它的机器。

1.2 构造性定义的判据

本文研究一类特定对象：能够吸收开放任务所产生的能力，同时又允许我们对长期运行与结构变化作出主张的 agent 机器。相对于宿主契约 Ω ，这样的机器负有四项定义义务：

1. **宿主边界。**所有能够影响所主张性质的机制，要么属于机器，要么被明确指定为 Ω 提供的输入或宿主假设；不存在未声明的第三处。边界同时决定主张的范围，以及性质应归属于何处。
2. **构造语言。**合法的机器形态由有限原语和有限组合规则生成。新能力的产生可以没有预定上界；同一门语言决定它们何时成为机器的一部分，以及以什么形态持续存在。
3. **合法转移。**初始机器以及改变其构成的操作都被显式规定。每一次合法转移都把一台合法机器映回同一个机器类。合法状态与合法转移共同构成机器的操作语义。
4. **规则遗传。**若要把自我改进归于机器，产生合法后继的规则本身也必须可表示、可构造并可遗传。当前规则在一次构造期间保持不变；修订后的规则被写入后继。这种时间上的分离避开了在同一瞬间构造自己的恶性循环。

前三项义务定义机器的载体与动力学；第四项把产生这些动力学的规则放进代际关系。若一个系统把加载器或构造服务明确划入 Ω ，它仍可在前三项义务下得到良定义；只是其构造能力属于宿主，第四项不成立。

这些义务定义的是一个机器类，而不是唯一架构。任何满足它们的系统都是这个类的成员。本文给出存在性见证：Dalek 是其中一台完整运行的机器。

1.3 必要性与治理面

四项义务都来自无界主张所要求的证明形式。令 P 表示“这仍是一台合法机器”或“这台机器仍携带构造后继所需的机制”。要在任意长的合法历史上证明 P ，一个有限论证必须给出基例，并证明每个合法操作都会把合法机器映回同一个类。归纳对象必须有确定的论域，这要求宿主边界与构造语言；归纳步骤必须能够无限复合，这要求合法转移；当形成归纳步骤的规则自身也进入变化域时，还需要规则遗传。

开放任务本身只要求生成性的有限手段：一个固定的通用解释器已经能处理未被事先枚举的程序。只有当生成出的能力还必须被装入、持久化、重启并遗传——即任务轴与自修改轴相交——构造语言与合法转移才成为机器的义务。同样，一个将规则保持在外部的系统也可以得到良定义，只需把这些规则声明为 Ω 的一部分。只有当改进被归于机器自身时，规则遗传才成为必要条件。因此，所得结论是一组关于机器类的义务，而不是一种唯一实现。

构成边界还带来双向收益。如果机器与环境之间的一切组织性作用都经过有限接口，监控与干预只需覆盖一个有限的治理面。环境不能绕过合法转移改写机器，而机器的组织性作用也获得确定的授权、审计、撤销与终止点。结构变化要么保留这些中介点，要么将其改变记录为一次显式转移，因此治理能够跨世代延续。

本文的主张限于组织性作用：成员、消息、门以及构成的变化属于机器。Actor 在一次调用中直接访问文件或网络，并不进入机器历史 (§3.2.3)。若要把同一判据扩展到所有物理效应，就需要一个更强的 Ω 。

1.4 Dalek：构造与见证

Dalek 由三个来源构成。冯诺依曼 1948 年的自复制自动机提供可遗传的构造核 [von Neumann, 1951]；四项义务要求显式边界、有限构造语言、合法的构成转移，以及进入遗传的规则；agent 底座的性质则使统一的 actor/message 接口与 channel 边界成为自然介质。三者结合产生的是一台新的 agent 机器，而不是冯诺依曼机器的一种新实现。 Ω 只提供执行、存储和网络。Actor 通过 message 协作，channel 提供组织边界。 G 描述机器应当是什么， H 记录它经历了什么。一个对功能与组织无感知的运行时 R 定义构成转移。构造器 A 、复印器 B 、控制器 C 与能力制造器 D 自身都包含在 G 中，因此当前世代可以构造一个携带修订后器官、乃至修订后运行时的后继。

这一构造把 agent 系统的组织中心从一次任务循环移到一个个体的生命周期。装入、重启、繁殖与升级不再是围绕 agent 进行的维护操作，而成为机器自身的转移。任务能力成为成员，并能在这一生命周期中生长、组合与繁殖。四个“自”只是同一生命周期的四个方向：维护、变化、延续与联合。

本文有三项贡献。第一，提出 agent 机器的构造性义务 (§§1.2–1.3)。第二，构造同时满足四项义务的 Dalek (§§2–3)。第三，以账本证据表明，同一结构能够交付自维护、自进化、自复制与自组织 (§4)。

下文安排如下。§2 陈述沿用的构造及其假设，并据此选择底座；§3 构造机器；§4 让账本逐行给出证据，并定义失败路径的语义和活性边界；§5 讨论这一定义的结果与范围；§6 回顾相关工作；§7 总结全文并提出保守扩展。

2 构造模型与底座

§1.2 的四项定义义务中，规则遗传已经有一个构造核：冯诺依曼 1948 年的自复制自动机 [von Neumann, 1951]。⁰ 本文保留这一构造核 (§2.1)，将它与其余义务结合，并在适合 agent 的底座上导出一台机器 (§2.2)。本章定义记号 A 、 B 、 C 、 D 与 G ，以及准静止和宿主契约。

2.1 沿用的构造

这一构造由四个部件和一份描述组成 [von Neumann, 1951]。 A 是通用构造器：给定描述 G ，它构造出 G 所描述的自动机。 A 保持固定；复杂度位于 G 中，并随待构造对象的复杂度增长。 B 是复印器：它不解释 G ，只复印它。 C 是控制器：它先指挥 A 按 G 构造，再指挥 B 复印 G 并把副本装入新构造，最后切离并启动后代。 D 是载荷：机器携带的任意自动机，但不参与构造与复印逻辑。完整机器为

$$E = (A + B + C + D) + G,$$

⁰通常所谓“冯诺依曼架构”，指 1945 年为存储程序计算机提出的算术、控制、存储、输入与输出组织 [von Neumann, 1945]。本文使用的是冯诺依曼在 1948 年提出的另一项构造——自复制自动机。前者组织一次计算，后者为一台能够构造并复制自身的机器提供构造模型。Dalek 以第二项作为构造 agent 机器时使用的内核。

其中 G 描述整个组合 $A + B + C + D$ 。 E 的一个周期产生 E 的逐字副本。¹ 同一份 G 被使用两次： A 解释它， B 复印它。 G 不包含对自身的描述；复印过程重新生成它，因此身体与身体的描述按确定的时间顺序形成 [von Neumann, 1951]。

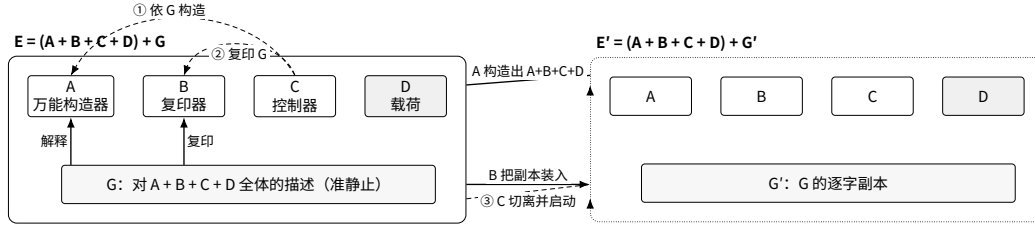


图 1: 冯诺依曼 1948 年的构造: $E = (A + B + C + D) + G$ 。同一份 G 被使用两次——由 A 解释, 由 B 复印。在 C 的指挥下, A 构造 $A + B + C + D$, B 放入一份 G 的副本, 随后新机器被切离并启动。 D 是不参与构造与复印的载荷。

这一构造依赖两个假设。第一, 描述是准静止的: 在复印期间保持不变。一个活的原件会在被检查时反应和变化, 因而不能被逐件复制; 一条在复制期间保持固定的线性链则可以 [von Neumann, 1966]。第二, 物理由环境提供。该构造是运动学的: 机器浸在零件之海中; 部件被公理化为黑盒, 只规定刺激与响应; 能量和运动属于环境 [von Neumann, 1951]。§2.2 给出这种环境契约的一种实现。本文选择的底座满足这两个假设。

构造模型与底座彼此独立。1948 年的构造以黑盒部件和环境契约陈述, 不依赖某一种具体的部件物理。1952–53 年设计的 29 状态元胞自动机只是后来的一种实现 [von Neumann, 1966], 并非构造本身。在遗传核这一层, Dalek 保留 1948 年的构造, 但替换其底座; 完整的 Dalek 架构还加入了边界、身份、历史与增长所需的结构。

以四项义务衡量, 1948 年模型提供了规则遗传的核心与构造语言的骨架, 但缺少三样东西。身份: 它无法回答“我是哪个个体?” [McMullin, 2000], §3.4.6 将给出答案。历史与增长: 它产生副本却不保留历史, 而复杂度的实际增长在后续文献中仍是开放问题 [McMullin, 2000]。Dalek 用账本记录历史 (§3.2.3), 并通过作者与验证者之间的内部回路让增长进入描述 (§3.4.3)。活的作者: 原模型的载荷不参与构造与复印, 且局限在载荷内的变异并不致命 [von Neumann, 1951]。Dalek 在同一位置放入能够写作的部件, 并保留记号 D 。围绕该部件的介质、法律与身份将在 §3 展开。

2.2 从构造模型到 Agent 机器

1948 年模型规定的是因果角色及其关系, 而非软件实现。它的构造需要五样东西: 可组装的零件、准静止且可复印的描述、由若干零件组成的器官、可切离实体, 以及提供物理的环境。此外, 它还在繁殖逻辑之外留下一个载荷位。

Dalek 将这个遗传核与 §1.2 的四项义务以及 agent 特有的底座性质结合。义务要求声明宿主边界、有限的构造与变化语言, 以及进入遗传的规则; 底座让异质参与者都通过文本与消息交换出现。二者共同导出以下组织。这些对应关系是构造一台 agent 机器的结果, 不是对冯诺依曼机器的逐项实现。

1. **零件 = actor**。Actor 是只通过消息交互的行为部件 [Hewitt et al., 1973]。所有构成性成员都经同一消息接口呈现; 程序、模型、工具、人和外部系统都被适配为 actor。其构造收益是统一: 零件只有一种表示, 所以 A 只需一条构造路径。
2. **描述 = G** 。在这一介质中, G 是文本: 静态、可操作、可打包的构成性数据。一旦写出, 蓝图本身就是介质中的一个部件。 A 的解释和 B 的复印无需跨越不同物理种类, 准静止也由此直接成立。
3. **器官边界 = channel**。Channel 把一组 actor 围在局部边界内, 并为它们维护一本账本。Channel 自身并不执行。²
4. **可切离实体 = Space**。消息介质没有内在空间, 因此“切离并启动”所指的实体必须由构造给出。一个 Space 由若干 channel 和一个运行时组成; 它是 Ω 能够启动的完整机器, 切离与启动以 Space 为单位 (§3)。

¹ 记法。冯诺依曼以 I 表示描述 (有载荷时写作 I_{D+F}), 以 D 表示组合 $A + B + C$, 以 F 表示附加载荷, 完整机器写作 $E_F = D + I_{D+F}$ 。Dalek 保留 A 、 B 、 C 的含义, 以 D 表示载荷, 并把描述写作 G 。冯诺依曼的 I 与 Dalek 的 G 是同一架构角色在两种介质中的名字。§3.4.7 列出全部记法与结构偏离。

² 构造器 A 、复印器 B 、控制器 C 与通用能力制造器 D 等器官以 channel 为组织粒度, 执行则由 actor 承担。为工程方便, A 、 B 、 C 三项功能位于同一个初始 channel; 它们共同构成执行一次初始构造所需的最小器官 (§3.4.1)。

5. 环境 = $\Omega + R$ 。Dalek 把 1948 年模型未分解的物理拆成两层。外部宿主契约 Ω 包含执行、存储和网络三种原始能力；内部运行时 R 执行介质法律，并随机器遗传。二者共同实现环境契约。
6. 载荷位 (Dalek 的附加要求) = D ，即通用能力制造器。 D 由大语言模型 L 与编译器 U 组成。 L 产生候选能力：它接收并产生文本，输出具有概率性，两次调用之间不保留可信状态，内部仍是黑盒。因此它原生的操作形态是收到一条消息，再发出零条或多条消息。 U 与作者分立：它编译候选，并把通过检验者接纳进机器。

由此得到的架构有三个相互独立的输入。1948 年模型贡献 G 的两次使用、构造器/复印器/控制器/载荷的划分、它们的时间顺序，以及构造期间的准静止。由 agent 问题导出的定义义务要求显式边界、构造语言、构成语义与后继规则。要让一个变化中的个体延续，还需要身份与历史；开放任务则需要实际增长的路径。可用部件的物理性质决定介质。计算机的组织服从元件物理 [von Neumann, 1945]；这台机器亦然。 L 所能感知与实施的一切都以文本流入流出，因此人、环境、工具与其他 agent 以同一种身份——actor——与 L 相遇，交互也都采用同一种形态——message。适配层被局部收进一个普通 actor 程序： L 的 loop 组装账本视图、调用远端模型、解析输出，再发出逐条 call。介质不为 L 设置任何专用语义。

这种适配关系可以直接列出：

Dalek 元件或结构性质	为什么适配 L
message / call：唯一构成性作用	文本是 L 与人类参与者共同的原生输入输出形态。
G (描述)：描述的两次使用与规则遗传	L 可以直接读写静态文本。
H (账本)：身份、恢复与事实见证	L 的概率性输出无法依靠重算恢复；历史将其保存。
U (编译器)：变异与选择分立	L 不应同时充当作者与裁判。
actor 无特权	L 可以被替换，也可以与另一个 L 并存。

3 机器

§2.2 的六项映射在本章逐项定义： Ω 与 R (§3.1、§3.3)，actor、message、channel、Space 与账本 (§3.2)， G 的解释、复印与遗传 (§3.4)。机器造完之后，§3.5 以 §1.2 的四项义务逐条核对。

3.1 宿主契约 Ω

Ω 是环境合同 (§2.2) 的界外一半：一份固定、有限、不含本文任何词汇的能力清单。机器是运行在 Ω 上的一个软件； Ω 不知道账本、器官、Space 与复制为何物。界内一半是运行时 R (§3.3)。

3.1.1 三条能力

能力	契约	第一个实例
运行 (Exec)	load(源码, 环境) → 可调用对象；spawn(源码, 参数) → 进程	python3：进程内 exec + 子进程
存储 (Store)	read / write / append(路径, 字节)；append 原子、落盘	文件系统
网络 (Port)	send(端点, 字节)；recv(端点) → 到达的字节序列	文件收件箱

三条是三个语义角色，合起来是一份充分且很小的合同；第一实例里接收侧由存储的文件追加实现。运行含进程，不止求值：源码成为独立存活、可被停止的进程；没有独立进程，子代不能在父代死后存活，自复制退化为函数调用。网络含接收，不止发送：人与机器对话、另一台机器递来描述、父代往子代的根门写行，都要求机器一侧先能收（单机实验中端点是文件收件箱；跨宿主时经网络）。存储含原子 append：介质的第一法律——只追加 (§3.2.3)——建立在这一条上。

3.1.2 边界

Ω 不提供权限、调度与消息语义、重演与验证、隔离；它们分别归机器的法律 (§3.3.4)、介质 (§3.2)、账本 (§3.2.3) 与目录约定。宿主边界 (§1.2) 在此取具体形态： Ω 的接口表上不出现本文的任何一个词；凡影响所证性质的机制，要么在机器之内，要么在这张表上。任何满足 §3.1.1 三条的底层系统都是合法的 Ω ，机器的身份不依赖特定宿主。本文实验所用的第一个 Ω 是 Linux + python3 + 文件系统；更换宿主时机器内随之改变的一层见 §3.3.5。

3.1.3 外生输入与时钟

接口表上没有时钟与随机数。凡穿过机器的组织边界进入介质、且不能由既有状态与确定程序重算的值，都是外生输入，一律入账 (§3.2.3)：人的消息、门送进来的字节、成员的回复、时间、失败、并发顺序，皆经同一个口——call 边界。actor 在运行内部从宿主取得的值 (L 的远端应答、读到的文件)，在被它写成 call、回复或外来消息之后才成为机器事实。于是每一步要么可重算比对，要么照账抄录。

时钟是同一原则的应用。机器的静止是准静止 (§2.1) 在运行态的对应：没有消息，没有任何东西动。体内带时钟的机器自己制造事件，静止从性质降为巧合。机器不产生时间，机器组织时间：需要 tick 的机器订阅一个膜外的 tick 源，tick 作为普通消息经门入账、带章。物理时间由此退出机器，机器内部只剩事件的因果序 [Lamport, 1978]。运行时每一步 run-to-completion，一条事件驱动一次调用 (§3.3.1)，不需要知道时刻。

3.2 介质：成员眼中的机器

本节给介质的语义——机器在其成员眼中的形状；§3.3 给驱动它的运行时。

3.2.1 三层膜：actor → channel → Space

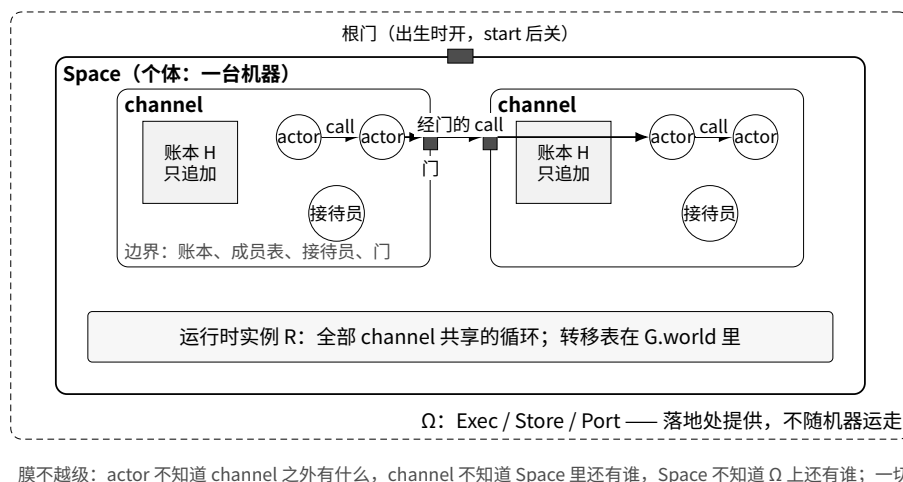
机器由三层套成，每层是一张膜。

Actor：行为。一个 kind、一段 text、若干绑定的句柄。actor 是机器里唯一做事的东西：收到一条消息，产出若干动作与一个回复（驱动方式见 §3.3）。text 是它的全部给定——程序的 text 是源码，门的 text 是地址。

Channel：边界。一本只追加的账本、一张已注册 actor 的表、一名接待员、若干扇门。channel 没有代码：它是纯结构，存在的只有各 actor 的 text；channel 的描述是机器描述中的一个结构节点。

Space：个体。诸 channel、门的拓扑、一份运行时实例。Space 是被 $\Omega.spawn$ 的最小单位：一个 channel 自己拥有的东西——账本、成员表、配置——全是静止的数据，令其动的循环在 Space 层被全部 channel 共享、不属于其中任何一个； $\Omega.spawn(channel)$ 没有入口。带着运行时的才是机器；本文说“一台机器”，指一个 Space。

膜不越级：actor 不知道 channel 之外有什么，channel 不知道 Space 里还有谁，Space 不知道 Ω 上还有谁；每一层对外部的全部知识，以消息的形式从边界进来（图 2）。



膜不越级：actor 不知道 channel 之外有什么，channel 不知道 Space 里还有谁，Space 不知道 Ω 上还有谁；一切作用只有 call。

图 2：三层膜。actor 在 channel 里，channel 在 Space 里，Space 之外是 Ω ；每个 channel 有一本只追加的账本、一张成员表、一名接待员与若干扇门；运行时实例 R 在 Space 层被全部 channel 共享；一切作用只有 call，跨 channel 的 call 经门；根门在出生时开、start 后关。

3.2.2 一切作用只有 call

成员对机器能做的事只有一件：call(地址, 正文) → 返回值。没有共享内存，没有全局变量，没有越过账本的旁路。地址空间如下：

地址	是什么	返回
tag (如 file、U)	本 channel 的一个成员	对方的回复
0	介质的读地址	账本行 / 成员表
门的 tag	出去的路 (§3.2.4)	空
syscall 与动词	写形态与动世界，需要绑定 (§3.3.1)	回执或拒绝

写给不存在的地址，消息被丢弃；写给成员，介质同步调用对方，对方的返回值 即 call 的返回值——嵌套的 call 构成调用栈，这是机器里唯一的控制流。

地址 0 的读不改变形态，对全部成员开放，不需绑定。0 认识两个词：show 返回账本的行——我经历了什么；who 返回此刻的成员表——我现在是什么。第三问“我应当是什么”由一个器官回答 (§3.4.2)。三问合起来，即 §2.1 所述原型答不出的那个问题在本底座上的着落：历史、形态、规范，各有一个可以 call 的地址。

以一个将在 §4 反复出现的成员为例。file 是一个程序 actor，正文约定是 read < 路径 >/write < 路径 >\n< 内容 >。某成员 X 在自己的一次运行中 call(“file”，“read notes.txt”)，账本上出现三行：

```
msg { seq k,    from X,    to file, body "read notes.txt", run j }
step { seq k+1, actor file, ..., run j }
msg { seq k+2, from file, to X,    body "<文件内容>", run j }
```

X 拿到返回值继续运行。它不知道 file 是谁写的、何时放进来的——地址在调用那一刻解析。file 是机器自己造的零件 (§4)，它的用户看不出这一点。

3.2.3 账本 H：入账即历史

每个 channel 一本账，只追加；一台机器的全部账本合称 H。行分四类：形态行 (place / retire) 记某地址上放入一个成员 (携带完整的 text) 或某成员退役——channel 此刻的形态由形态行折叠得出，即 who 的实现，也是重启的根据 (§3.3.2)；消息行 (msg) 记穿过 call 边界的请求、非空的回复与回执，记法按地址类别分 (§3.3.1 第 3 条)；机器醒来时 R 写给首 channel 接待员的 up 也是一条 msg；步行 (step) 记一次运行的收尾——起因的 seq、它发出过的全部 call、错误 (若有)；边界行 (down) 记一次合法停机的完成，只出现在首 channel 的账上 (§3.3.1)。

介质的前两条法律在此立字，其地位在 §3.3.4。

第一，from 由介质盖章。没有成员能替别人署名；膜外进来的消息，由介质署上对应的门。账本上的“谁说的”与“说了什么”同等可信。

第二，入账即历史。追加进账本的行就是已经发生的事，不撤销、不改写；一切可恢复的“当前状态”都是对历史的折叠，不是与历史并立的另一份记录。没入账的没有发生：actor 在一次运行内部做的事——局部变量、中间值、它与远端的往来——不在账上，不属于机器的历史；穿过 call 边界的才是。§3.1.3 的外生输入由此有了存放处：凡穿过边界的非确定值，账本原样收录，历史里没有“当时算出过什么”，只有“当时说了什么”。

同一条边界划开另一件事：成员在运行内部触碰宿主的世界 (读写文件、访问网络)，不是对机器的作用——不入账、不遗传。机器的膜是关于组织与账本的膜，不是关于计算的沙箱 (§1.3)。效果成为机器的效果，唯一的路是成为账本上的一行。

3.2.4 门：唯一指向外面的 kind

门是一种 actor，其 text 不是行为，是地址——门是全机唯一 text 指向 channel 之外的 kind。写给门的消息被原样送到 text 所指端点，署名由介质给出；门不产生输出、不解释内容，只搬运。整台机器的拓扑即各部门的 text 的集合。两个 channel 接通，即两本账各有一扇门互指；对面是本机器的另一个 channel、是人、是另一台机器，门的规则只有同一条。§2.1 中 $E = A + B + C + D$ 里的“+”，在这个介质里的定义即此：接上，就是两本账互相有门。

门与成员的区别不因实现相同而消失：成员是功能——收消息、产出本 channel 里的动作，去掉它机器少一个部件；门是连接——不产出、只搬运，去掉它机器少一条边。同一个远端，接成成员是它在回答，接成门是它在隔壁。

3.2.5 进来的路

外面的东西作用于机器，只有一条路：收件箱 → 接待员。每个 channel 有一个收件箱 (Ω 的接收侧)；投进来的东西被介质写成一行 msg——署名对应的门——投给该 channel 的接待员：形态表上带接待标记的那一个成员。“作用于一个 channel 的一切都在它的账本上”对膜外同样成立。

此外有一扇特殊的门：根门，Space 级，不属于任何 channel。根门开着，当且仅当所有账本上还没有任何消息行。开着时它接受两类行——造形态（建 channel、放成员）与第一条消息；那第一条消息既是这台机器的第一次心跳，也是关门的动作——此后根门的行一律被忽略，机器对世界只剩收件箱一条路。谁在根门开着时做了什么、第一条消息是什么，是出生 (§3.4.4) 的内容。根门只在出生前开。

3.3 运行时 R

原型里令 A、B、C 动的机制是底座的物理，不在机器里，不在 G 里 (§2.1)。底座换掉之后，这份物理的承接者即运行时 R：消息的投递、成员的调用、账本的落笔。

R 不是 Ω ： Ω 不认识 channel，R 全部的工作就是驱动 channel；空白宿主上它随机器一起到达。R 也不是机器的某个成员：它驱动所有成员，包括承担构造的那些 (§3.4)。R 的住所是机器描述的一个根字段 world (§3.3.5)：随每台机器遗传，不含任何一台机器的信息。对机器而言 R 是物理：成员的介质接口上没有读或改当前 R 的动作，每个成员生来浸在其中；对 Ω 而言 R 是软件。环境合同的界内一半即此 (§2.2)。

R 的定义：一个极小的状态空间，加一张对内容重命名不变的转移表。它有操作语义（什么东西移到哪里），没有意图语义（这一步是在构造还是在聊天，它不知道）。

3.3.1 描述语言、状态与转移

合法 G 是 A 的受理域 (§3.4.1)：realize 能完整构造的描述。其语法：

字段	内容	约束
world	ω -bind、loader、R 三段文本	不含本机的任何信息 (§3.3.5)
channels	有序表，每项为名、成员表、接待员	名在机内唯一；首 channel 必有接待员
成员	$\text{kind} \in \{\text{program}, \text{door}\}$ 、 text 、 $\text{bind} \subseteq$	程序的 text 是源码，门的 text 是端点；能否实例化、是否规格不属于语法 (§3.3.4、§3.4.3)
peers	$\{\text{syscall}, \text{spawn}, \text{stop}\}$ 、tag channel 名对	每对在两端各生一扇门

合法 G 是描述语言。机器状态中的 G_t (§3.4.2) 是这门语言写成的一份数据，在不在语言之内是状态上的一项性质 (§3.5)。

R 的可恢复状态：每个 channel 一本只追加的账本，每个 actor 一个游标（处理到哪条事件），每个收件箱一个偏移（读到哪一行）；三者都由账本折叠重建，成员的易失状态 Σ 不在其中 (§3.3.2)。事件：某本账上多了一条不带 run 标记、写给某个地址的 msg 行。

转移，R 应答的全部：

1. 收件箱到达一行 → 写一条 msg 给该 channel 的接待员，from 为指回发信端点的门。
2. 根门开着（所有账本无 msg 行）→ 接受写形态的 syscall 与第一条 msg；第一条 msg 落账即关门 (§3.2.5)。
3. 事件 → 调用地址上的成员一次，调用跑完才处理下一条 (run-to-completion)；调用结束记一行 step 并推游标，异常记入 step。机器没有别的并发源。调用内的每次 call 按地址记账：成员地址记一行请求，非空回复再记一行；地址 0、syscall 与动词各记一行事实或回执；门返回空；不存在的地址不记 msg 行，只留在 step 的调用帧中。
4. 写形态的 syscall → 形态行；动世界的动词 → 宿主动作（下表）。
5. stop → 当前调用结束后写一行 down，R 退出。
6. 醒来 (Ω .spawn 起 R，账本上已有 msg 行) → 折叠全部账本，写一条 msg up 给首 channel 的接待员；游标未推过的事件再次投递。
7. 进程被杀 → 没有 down；下次醒来时 start 或 up 之后无配对的 down，即上次为异常终止。

actor 是常驻函数：折叠到它的 place 行时实例化一次，挂在地址上，活到退役；此后每条消息都是调用同一个对象的 run(m)，返回值就是回复。call 在实例化时注入一次——对一个成员，它是常量。kind 有两种：

kind	实例化（放入时一次）	每条消息
程序	Exec.load(text, {call, me, channel}): 源码跑一次, 定义 run	run(m); 返回值 = 回复
门	— (text 是端点地址)	Port.send 原样送出, 介质署名; 返回空

写形态与动世界的地址是 R 直接应答的请求, 须由 G 中的 bind 字段授予; 无绑定者发出即被丢弃:

组	请求	R 的动作	回执
写形态 (syscall)	channel.create 名 channel.add.actor 名 kind text [bind] [tag] channel.retire.actor 名/tag	新建账本与收件箱 分配唯一 tag, 写 place 行——完整 text 入账 写 retire 行	名 channel/tag channel/tag
动世界 (动词)	spawn 包 stop	Ω .spawn(包): 起一个 新 Space 当前调用结束后写 down, 本 Space 退出	进程句柄 —

没有第三种 kind。L 也是程序——text 是它整个 loop（端点、提示语、组装账本 视图、把输出拆成帧 逐条 call、收返回值接着说），介质看到的只是一串 call 和一个回复。R 不知道 python，不知道模型； Ω 里没有为 L 专设的东西。

3.3.2 折叠与重启：恢复形态，不恢复状态

channel 此刻的形态不是另一份记录，是对账本的折叠：把 place 行与 retire 行 从头放一遍，得到成员表——§3.2.2 的 who 即此折叠。重启因此不需要任何账本 之外的东西：重启 = 重新折叠 = 全部重新实例化。醒来由 R 写给首 channel 接待员的 up 启动；游标未推过的事件再次投递，重复效应由成员据自己的账本消除。

每个常驻函数在两次调用之间可以持有内部状态 Σ （变量、缓存、未写完的念头）； Σ 不入账（§3.2.3：账本记 call 边界，不记运行内部），重启后 Σ 归零。活着的机器是 (G, H, Σ)，个体是 (G, H)——同一台机器，不保证同一个易失 现场。这条轴是恢复形态，不恢复状态：重启按可折叠的 H 重建实际形态、重新实例化仍在形态中的成员，不保证回到崩溃前那一刻，机器是否仍可运行、可构造、可繁殖是另外的性质（§3.5）；要活过重启的成员，自己 call(“0”, “show”) 从历史里把自己重建出来——策略在它的 text 里，不在 R 里。

3.3.3 R 对功能与组织无感知

R 只处理形，不处理义。它读的东西有且只有三样——地址、kind、text 作为该 kind 的参数。词汇表分两栏：

R 认识的（介质词汇）	R 不认识的（组织词汇）
地址、tag、门	构造器、登记处、作者
kind：程序、门	谁有权放成员、放了算不算数
消息、账本、追加、投递、步、回执	realize、pack、decl
text 作为参数	text 的含义

检验可执行：把 G 里所有组织层的名字整体替换（c0 改叫 x7，源码等价改写），R 的行为不变，两份历史在重命名下同构。违反的形态可以列举：R 里出现 if name == “c0”；R 认识一种“构造请求”并替成员执行它；R 校验“这是不是一台合法机器”；调度器偏心某个成员。每一条都把待证的事——构造由机器自己完成——放进了物理。承担构造与登记的成员（§3.4）的特权由此是组织上的：谁绑定了 syscall，写在 G 里，R 照 G 绑；在物理上它们与任何成员平等。

3.3.4 法律及其地位

介质的法律共三条，前两条已在 §3.2.3 立字：

1. 只追加，且 from 由介质盖章。历史不可改写，署名不可伪造。
2. 一切作用只经消息。没有成员能绕过账本触到另一个成员。
3. 步公平。R 持续运行、目标未退役、每次调用终止且未 stop 时，已入账 的消息终被投递；R 不偏心、不丢弃、不重排。

它们是法律：住在 R 里，R 对个体是物理。任何成员——包括作者——的全部作用 手段是 call，call 本身由这三条定义；违法在这个介质里不可表达。§1.3 的 有限治理面即此：与机器组织有关的作用全部经过 call，授权、审计、撤销与停 止各有确定位置。

法律的边界同样立字。中途的一律丢弃：一次调用跑到一半崩溃，未落账的一 切（ Σ 、局部变量、没写完的行）不属于机器；已落账的部分效应则是历史——机 器可能因此处在形态被改了一半的状态，这是损伤，不是矛盾，修复归器官（§3.4；§4.6）。未定义只在介质之外：H 不再是 R 能折叠的账本，或宿主违 反 Ω ，描述这台机器所依赖的物理不复存在。R 因此不需要事务、不需要两阶段提交：它只保证账本是 真实 的历史。再投递，不回放：折叠只重建形态；游标未推过的事件由 R 再次投递，已 落账的部分效应 是否重复、如何消除，由成员据账本处理；整机历史的全量回放不 是本模型的选择（其代价见 §4.7）。

3.3.5 world：对个体冻结，对谱系可变

机器描述 G 有一个根字段 world，装三样东西： ω -bind（ Ω 契约在本宿主上 的实现）、loader（机器的 目录布局与启动协议）、R（转移表本身）。world 的判 据是一张两问表：

	随机器运走？	含这台机器的信息？
Ω	否——落地处提供	否
world	是	否——同版本世界，每台机器逐字相同
G 的其余部分	是	是

介质上没有改变当前 world 的转移：成员抄它而不读， Ω 执行它，R 不读 G。world 于是对个体冻结。对 谱系可变：作者可以写出 world'，装进子代的描述，让子代用新物理出生，以不动点测试验收（§4.4）。 器官的节奏与此不同：c0、c1 的成员可以代内 add 与 retire，替换完成后立即用于下一次繁殖；一次具 体的 构造使用的是已装好的器官与一份固定的 G 快照（§3.4.4）。

原型的 G 不含转移规则，规则表属于底座（§2.1）。本文的底座是文本：world 随每台机器运走、随谱系 演化，物理本身进入谱系。规则遗传（§1.2）在此落地。

3.4 器官、遗传与出生

§2.1 的四个角色落到 channel 上（§2 脚注 3）。c0 承担 A、B 与 C：realize 解释 G（A），pack 复印 G（B），编排流程（C）。c1 是登记处：描述 G 的居所。原型里描述是一条静止的带；本文的描述由 c1 从 自己的账本折出（§3.4.2），c1 是原型之外的器官，§2.1 所述“增长进入描述”的位置。登记处独立于构 造器，因为规范形态与实际形态必须是两个可比的对象（§3.4.2）。粒度上，A、B、C 合居 c0，登记处独 占 c1，口径相同：一次构造所需的器官放在一起，被比对的对象单独成账，G_t 因此恰是一本账的折叠。 准静止属于 decl 返回的那份 G 快照：它在 pack、spawn 与 start 期间不变（§3.4.4）。c2 承担 D。器官 是组织，不是物理：R 不认识 c0、c1、c2，它们的全部特权是 G 里的几个 bind 字段，R 照 G 绑。c0 之 所以是构造器所在，只因为会 realize 的成 员注册在其中——换个名字、挪到别的 channel，机器照跑。

最小机器 {c0, c1} 已经闭合复制：它的描述只能被抄，不能被扩展，因为它没有 作者。加上 c2 才是完 整的机器：能维护、能遗传、能变异出新能力。本节逐个立 起三个器官，然后讲出生、繁殖与身份。

3.4.1 c0 = A + B + C：唯一解释 G 的地方

构造分死活两半。pack(G) → P 是死的：把 G.world 写成文件、G.json 原样 放旁边——不解释 G，不 生成任何东西，同一份描述永远同一个包（纯函数）；这是 B。realize 是活的：运行中的 c0 读 G 的 结构，逐项发 syscall——经根 门造子代，或在本机器内生长；这是 A。全机只有这一个地方解释 G；其 余每一双碰到 G 的手都在抄。

realize 的核心操作只有一个：按一个 channel 描述造一个器官。造一台机器 = 依次造它的器官，再按拓 扑放门。channel 从不被单独复制——“复制一个 channel”就是用同一份描述再造一个器官，仍是构造。

A 与 B 的分离在此落到字段上。描述分两类字段：结构（有哪些 channel、每个成员的 kind、接待员、 门指向哪）由 realize 解释；text（每个成员做 什么）realize 只抄不读，日后由 R 解释。c0 搬运一段源 码时不知道里面写了什 么——A 读结构，不读内容。

通用性落在 *realize* 的受理范围上：它对任何合法 *G* 工作，不只对自己的。只复印自己的机器不构成 *A*。
C 是 *c0* 里的另一个成员：编排 *decl* $\rightarrow$ *pack* $\rightarrow$ *spawn* $\rightarrow$ *build* $\rightarrow$ *start* 的普通程序，绑定动世界的动词；全流程见 §3.4.4。

3.4.2 c1: 登记处, 与形态的分裂

复制要求描述准静止 (§2.1)，而机器活着——运行中的形态每时每刻被消息改变。此矛盾的解，是让实际形态与遗传形态分裂：实际形态 *M_t*, *R* 对全部账本的折叠，此刻活着的结构；遗传形态 *G_t*, *c1* 对自己账本的折叠，这台机器认为应当遗传与再生的结构。

c1 是注册了登记员的 *channel*。它只折自己的账本；账本里是 *c0* 经门送来的三种登记——*born* (经根门放的)、*placed*、*retired* (*c0* 的手做的)——于是 *decl* = *G₀* $\oplus$ *placed* $\ominus$ *retired*。*c1* 存的是 *c0* 的承诺，不是 *R* 的事实：*R* 记的是“我现在是什么”，*c1* 记的是“我应当是什么”。

两者的关系是一条要求：静止时 $\pi(M_t) \cong G_t$ ，其中投影 π 按出处定义——保留构造器放的一切与经根门放的 *c0* 成员，去掉出生证明、临时门、退役的成员。这条不是 *R* 的不变量，是 *c0* 维护的性质：持有 *syscall* 的器官能造真的也能记假的，登记被写歪就是一次变异，由后代的存活淘汰——膜划的是机器与世界的边界，不隔离机器与它自己的器官；自我修改的能力即在此。形态改动途中、损伤之后，*M_t* 与 *G_t* 暂时不合。自维护在本模型里的操作性定义：相对于修复开始前已固定的目标 *Q*，机器经自己的构造路消除偏离或恢复 *Q*——形态上 *Q* = *G_t*，即把 $\pi(M_t)$ 拉回 *G_t*；功能上 *Q* 是成员的规格；能力上 *Q* 是可运行、可繁殖等性质。*Q* 由 *G*、既有历史或膜外递入的规格给出；目标的自主生成与内部持有不在本文之内。³失去完整、失去繁殖能力、*G_t* 被写出描述语言，都是合法转移可达的状态，机器在其中仍有 *G_t*、*H* 与可折叠的形态。回去的路径可以不存在：修复所经的器官也损坏时，机器停在一个定义良好而不可恢复的状态，仍带着身份与历史 (§4.7)。

由此得到环 *G_t* $\rightarrow$ *M_t* $\rightarrow$ *H* $\rightarrow$ *G_{t+1}*：描述被 *realize* 展开成活的形态，形态的活动写进 *H*，其中经 *c0* 登记的部分进入 *c1* 的账，*c1* 把它折回下一份描述；未经登记的历史留在 *H*，不进 *G*。描述与形态在这个环上互相定义，环的每一段都在账本上可查 (图 3)。

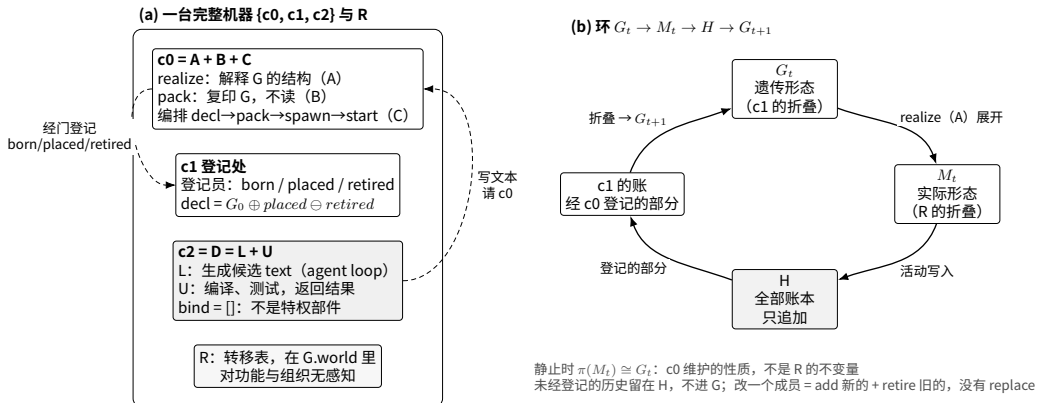


图 3: (a) 一台完整机器: *c0* 承担 *A*、*B*、*C*, *c1* 是登记处, *c2* = *D* = *L* + *U* 的绑定为空, *R* 的转移表在 *G.world* 里; *c2* 改形态与膜外走同一条路——写文本、请 *c0*。(b) 环 *G_t* $\rightarrow$ *M_t* $\rightarrow$ *H* $\rightarrow$ *G_{t+1}*: 描述经 *realize* 展开为实际形态, 活动写入 *H*, 经 *c0* 登记的部分进入 *c1* 的账, *c1* 把它折回下一份描述; 静止时 $\pi(M_t) \cong G_t$ 由 *c0* 维护。

改动的语法也在此定下：没有 *replace*。*place* 行不可改写 (入账即历史)，故改一个成员 = *add* 新的 + *retire* 旧的，新的拿新的物理地址，逻辑 *tag* 可由 新件接任；退役是追加的一行，不抹除任何历史。历史里它永远在，形态里它没了。

³“已声明的目标”有两层：规格——机器应具备的能力（构造、复制、组网、繁殖）及其验收条件；描述——各成员的具体实现文本。*G* 只承载后者；规格在本文中是隐含的，未被明确定义——它存在于 §2.1 的四角色架构、任务文本与实验者的判断中。规格因此从膜外经门递入：§4.5 中 *C* 的恢复所依据的是递入的协议规格而非 *G*，重写的 *C* 与原版同规格、异实现；§4.7 活性条件的第四条说的也是这件事。膜外递入是规格在机器内尚无位置时的替代形式。把规格收进机器不需要新机制：持有规格的成员与递入规格的在介质上是同一种东西——一个成员、一条消息，差别只在规格是否随 *G* 遗传。把规格定义为独立于 *G* 的一等对象——规格定义完整性，*G* 是满足它的一种实现，自维护对规格陈述——留待后续工作。

3.4.3 $c2 = D = L + U$: 作者不是特权部件

最小机器造不出新成员，因为它没有作者。 $c2$ 补上这一格： L （生成——它的 `text` 就是整个 `agent loop`：组装账本视图、问远端、把输出拆成逐条 `call`）与 U （编译——把候选 `text` 变成可装入机器的部件；第一实例收到候选源码与测试，用与 R 同一个 `exec` 在进程内实例化候选，跑测试，返回结果）。“通用”属于 L 与 $Exec$ ：能算什么由 $Exec$ 给定，能写出什么由 L 给定， D 不增加二者之一。 D 制造的是成员——把 L 与 $Exec$ 已经能做的事，构成化为机器的成员：有名字与地址，落在 H 与 G 中，任何成员可调用，重启后重新实例化，繁殖时逐字继承，换代时 `add` 新、`retire` 旧。机器的能力增长即成员的积累，不是底层计算能力的增加。

两者缺一即不闭合。 L 单独产生描述，但机器没有任何内部判据分辨“能跑的”与“看着像能跑的”——可靠性成了对作者的假设，不是机器的性质。 U 单独是一个解释器，机器本来就有，它不产生任何新东西。合起来才闭合一个环：提出 → 检验 → 改 → 装，即变异与选择的内环进入膜内：能不能跑，机器自己判；值不值得要，仍在膜外。自进化在本模型里的操作性定义：一次经选择的变化进入 G ，并沿谱系遗传。它与自维护正交：恢复 Q 的那次变化若以新的实现进入 G ，一次运行同时是二者 (§4.1)。新东西的来源在机器外 (L 的远端、人)，生产新描述的组织的机器里——机器不产生随机性，机器组织随机性，与时钟同一原则 (§3.1.3)。

U 的边界：必要，不充分。候选在 $c2$ 的进程里，拿作者的 `call` 与作者的能力被测试；装进目标 `channel` 后，它面对另一套邻居、另一组绑定。内环筛掉“根本跑不起来的”，筛不掉“在别处才错的”（实例记录见 §4.3）。膜外的选择省不掉。 U 与 R 共用同一个 `exec`：机器里的编译器就是机器的物理 (§6)。

作者不是特权部件。 L 与 U 的绑定都是空的 (`bind = []`, G 里可查)； $c2$ 要改机器的形态，与膜外的请求者走同一条路——写文本、请 $c0$ 。故 D 可替换、可并存两个、子代可以带一个不同的 D ； L 能写出新的 L ，经 $c0$ 装入、随 G 遗传——`agent` 的 `loop` 本身是可遗传、可变异的形态，介质定死的只有 `run(m)` 与 `call`。载荷位 (§2.2) 坐进了作者，座位本身也在 G 中。

3.4.4 出生：封闭性从不变量变成一次事件

一台机器如何开始。没有按 G 预装的 `boot`： $\Omega.spawn$ 起 R ，根门开着，账本全空，机器静候。谱系的起点是人：第一台机器由人以同一套 `syscall` 经根门造出，人的手临时承担 A 、 B 与 C ——发 `start` 关门的那一步即 C ；此后每一代由机器内部的器官完成。流程六步，两边账本各记一半：

1. 父代取 $G = c1.decl()$;
2. C 打包： $P' = pack(G)$ (B : 抄，不读)；
3. C 起进程： $\Omega.spawn(P')$ ——子代的 R 起来，根门开，等；
4. 父代的 `realize` 经根门只造 $c0$ ：建 `channel`、放 $c0$ 的每个成员 (`text` 逐字搬运)，最后放一扇指回父代的门 (出生证明，不在 G 里)。子代账本上这几行 `by=_root`；
5. C 经根门发第一条消息：`start\n<G>`——正文带着 G 。门关，切离，父代义务结束；
6. 子代的 `realize` 收到 `start`，用本地 `syscall` 长出其余全部 `channel`、成员与连线 (`by=自己`)。发育完成，静止，等收件箱。

三件事在这份流程里同时成立。其一，父代从不进入子代的进程，它只往子代的根门写行；子代账本上看得见两只手——经根门放的与自己长的。唯一不是 $c0$ 造的器官，是 $c0$ 自己。其二，切离 = 关门 = `start` 是同一个动作：门开着时，形态由膜外决定，机器是被造的对象；门一关，形态只能经自己的 $c0$ 改，机器成了主体。封闭性由此从一条要守护的不变量，变成一次事件——出生。其三，两件事分开成立：子代在 `start` 之前账本上没有 `msg` 行 (§3.2.5 的根门规则)，没有任何东西动；`pack` 取的 G 快照在复制期间不变。准静止 (§2.1) 在此成立。

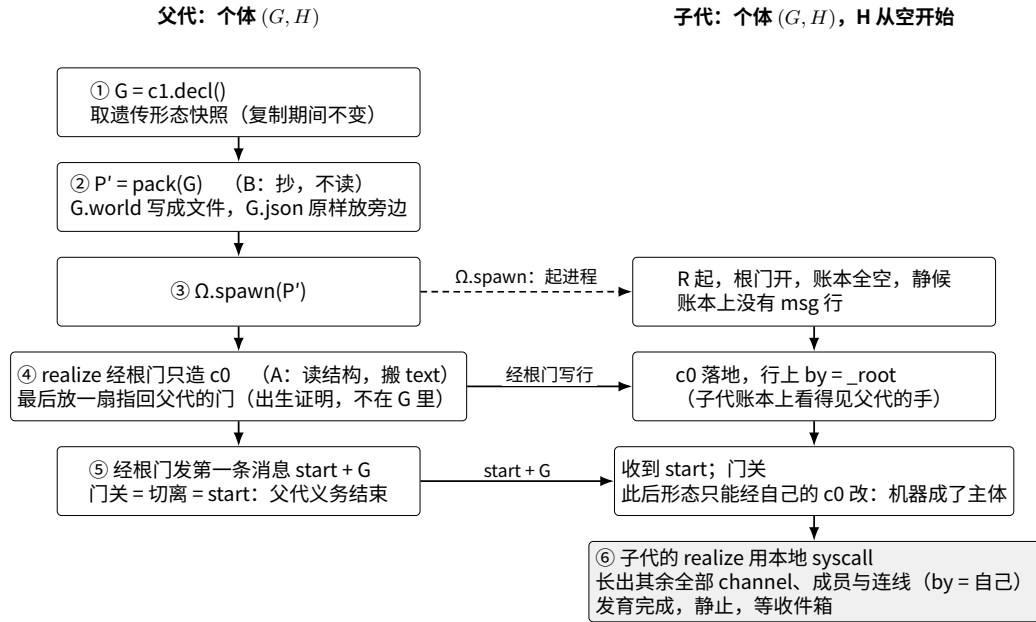
3.4.5 繁殖：同一份 G 的两次使用，落在字段上

繁殖三步 `decl` → `pack` → `spawn` 已在出生流程里全部出现。 G 的两次使用 (§2.1) 在本底座上是两类字段的两种待遇：`pack` 抄整份 G (B : 不读)，`realize` 读结构、搬 `text` (A : 解释组织，不解释内容)。 B 不给构造器开后门： $c0$ 、 $c1$ 的源码就是 G 里的两段 `text`，`pack` 从描述里抄它们进包，不从自己运行中的代码取——“描述被用两次”落在构造器自己身上，任何人都可以对包检查描述有没有被吞掉。

`pack` 的输出是死的 (源码，不是进程)： A 造出静止的机器， C 才启动 (§2.1)。死的才能跨宿主 (进程传不过网络，字节传得过)，死的才能比 (不动点判据比的是两份描述；§4.4 靠它)。复制在两边都可重演：父代账本上有发出的每条 `syscall`，子代账本上有落地的每条。

3.4.6 身份：个体 = (G, H)

§2.1 所述原型之外的第一样在此给出。个体 = (G, H)： G 答“我是什么”， H 答“我是哪一个”。 G 可复制——它是准静止的描述，复制正是它的用途； H 不随繁殖继承——子代以空的 H 开始。两台 G 逐字相同的机器，形态无从区分，历史从第一行起分岔。§3.3.2 已给出这条定义的动力学半边：重启后 Σ 归零而 (G, H) 俱在，所以还是同一台机器。



两边账本各记一半: 父代账本上有发出的每条 syscall, 子代账本上有落地的每条。
 G 的两次使用落在字段上: $pack$ 抄整份 G (B), $realize$ 读结构、搬 text (A)。
 子代继承 G (含 $world$; 作者可换入 $world'$, §4.4), 不继承父代的 H 与宿主侧的文件。

图 4: 出生的六步, 两边账本各记一半。父代: $decl$ 、 $pack$ 、 $\Omega.spawn$ 、经根门只造 $c0$ 、发 $start + G$ 并关门; 子代: R 起而静候, $c0$ 落地 ($by = _root$), 收到 $start$ 后门关, 自己的 $realize$ 长出其余全部 ($by =$ 自己)。子代继承 G , H 从空开始。

§3.2.2 的第三问也在此还清。自省三样至此配齐: $show$ 给本 $channel$ 的历史 (我经历了什么), who 给本 $channel$ 此刻的形态 (我现在是什么), $decl$ 给全机的遗传形态 G_t (我应当是什么); 全机的实际形态 M_t 是 R 对全部账本的折叠, 不是某个地址的回答。原型答不出的那个问题, 在这个底座上是三个可以 $call$ 的地址。回答没有用到任何新公理, 只用了账本的存在。

名字是推论, 不是登记。机器不需要外部命名机构: 子代就是父代账本上那次繁殖事件——它的 P 在父代包的 $spawn$ 之下, 孙代嵌套其中。这个名字是谱系内的局部名: 同一谱系里唯一, 两个始祖各自的子代可以同名; 身份仍是 (G, H) , 名字由账本推出。以宿主级计数命名子代时, 两台机器一合并即撞名——身份不在描述与历史里, 就会从宿主漏进来。

3.4.7 与原型的偏离清单

保留了哪些、替换了哪些、各换来什么:

	原型 (§2.1)	本文	换来什么
保留	$A / B / C$ / 载荷四分; 描述两用 (解释一次、复印一次); C 的启动与切离; 构造期准静止	逐条保留	架构即 §2.1, 无发明
替换	A 逐格施工 (构造臂置元胞态)	$realize$ 照 G 发 syscall	图纸即零件 (§2.2), 构造臂坍塌为折叠规则
替换	通用性 = 任意自动机 (元胞自动机内)	通用性 = 任意合法 G	通用性相对介质的合法描述空间陈述
替换	底座 = 29 态元胞自动机	环境合同 = $\Omega + R$: Ω 显式, R 随机器遗传	底座可换, 机器身份不依赖宿主 (§3.1.2)
替换	父代的 A 造出完整子代	父代只造 $c0$ + 复制 G + 发 $start$, 其余子代自己长	父代义务最小; 子代的构造器出生即被检验

	原型 (§2.1)	本文	换来什么
新增	— (描述是一条静止的带)	c1: 描述由账本折出的登记处	增长进入描述 (§3.4.2)
新增	— (没有历史)	H: 只追加、盖章的账本	身份、自省、恢复 (§3.2–§3.3)
新增	D = 不动作的载荷	D = L + U, 作者	变异从外部噪声变成机器的器官
新增	转移规则不在描述中	world 在 G 里遗传	物理本身入谱系 (§3.3.5)

架构来自原型 (§2.1); 账本、身份与法律来自定义义务 (§1.2); 登记处与 D 在已成立的介质上闭合增长与变异的环。各条的实测见 §4。

3.5 机器与判据

四项义务 (§1.2) 在上述机器上的形态如下。

宿主边界。账目分四处。机制在 G 与 Ω : G 装 world 与各成员的 text、bind、接待员标记, R 在 G.world 中, 器官的特权是其成员的 bind 字段; Ω 表装三条能力, 表上没有本文的词。事实在 H: 性质的见证是账本上的行。易失的执行现场在 Σ : 它影响成员的当前行为 (一个成员的计数器重启后归一), 身份、恢复与跨代遗传的命题不以它的保存为前提 (§3.3.2)。凡影响所论证性质的机制, 要么是 G 的一段文本, 要么是 Ω 表上的一条能力, 没有第三种位置。

构造语言。两种 kind 与三条写形态的 syscall 生成机器的全部实际形态——R 只接受这两种 kind, M_t 里没有别的; 描述语言合法 G (§3.3.1) 由同一组原语写成。新能力可以无界产生——L 的输出没有清单; 它进入机器分两个时刻: place 行落账, 进入实际形态 M_t; placed 登记进 c1, 进入遗传形态 G_t。两个时刻之间可以崩溃, M_t 与 G_t 由此分裂 (§3.4.2)。持久的形式在两处都是一段 text。生成的一侧无界, 装入的语言有限。

合法转移。合法状态由介质法律划定 (§3.3.4): 账本只追加, from 由介质盖章, 行属五种, 形态行的 kind 属程序与门两种, 当前接待员不被退役; 满足这些的一组账本, R 都折叠得出形态。初始机器是创世所得的第一个合法状态 (§3.4.4)。合法转移是 R 的七条 (§3.3.1)。保持由 R 的位置给出: R 是账本唯一的写者, 七条转移写下的每一行都满足法律; 成员改变构成的唯一手段是 call; 无绑定者的写形态请求不进入 R 的动作, 有绑定者的违法请求以拒绝回执落账, 两者都不落形态行——成员的任何作用都产生不出非法状态, 违法不可表达。归纳即得: 从初始机器出发、经合法转移到达的每个状态都是合法状态。这个类由介质法律划定, 不含完整性: 完整、可运行、可构造、可繁殖、G_t 在描述语言之内, 是合法状态上的性质, 可由合法转移失去 (§3.4.2), 其维持与恢复归器官 (§4)。

规则遗传。产生合法后继的规则分三层, 都在 G 中: world 是介质的转移规则; c0、c1 的 text 是构造、复印、控制与登记的规则; D 的 text 是变异产生与候选选择的策略。前两层定义结构的合法性, D 决定搜索什么变化。三层皆可表示 (文本)、可构造 (realize 只搬运它们)、可遗传 (pack 逐字复印)。替换节奏分两种——world 对个体冻结、在后继中生效, 器官的成员代内可换 (§3.3.5); 一次具体的构造使用已装好的器官与一份固定的 G 快照, 同一时刻构造自己的循环由事件次序消除 (§2.1)。

4 运行中的机器: 从一个任务到第三代

§3.5 把机器的性质分为两层: 介质合法由 R 的转移保持, 完整、可运行、可构造、可繁殖由器官维护。本章是器官层的见证, 分五类轨迹: 能力的构成化与遗传 (§4.2)、种群的组织与修复 (§4.3)、规则的跨代替换 (§4.4)、成员的丢失与恢复 (§4.5)、崩溃、停机与损伤 (§4.6)。每类轨迹改变的是不同的性质; 四个自是同一台机器上可交叠的性质, 各在其中有一段可指认的行, 一次运行可以同时落在几个自上 (§4.1)。坐在 D 座位上的是外部通用模型 (型号随实验注明); 任务、诊断、tick 与 spawn 请求经门递入, 人不越过门直接改写机器组织; 故障注入——硬杀进程、删除账本——发生在宿主侧, 是 Ω 上的事件, 不是机器的转移 (§3.3.1)。“机器做到某事”指账本上的行, 不指作者的自述。⁴

4.1 四个自

四个自各有一个操作化定义与一句验收句:

⁴本章所引账本的原件与本文所附快照, 对照见附录 B。E1、E2 的原始账本生成于 tag 寻址之前, 外部回执使用数字地址 (如 c2/5); 现行协议用 channel 内唯一的 tag (如 c2/file, §3.3.1)。原始记录不改写, 本章照录。

	操作化定义	验收句	见证
自维护	相对于修复前已固定的目标 Q (由 G 、既有历史或膜外规格给出, §3.4.2), 机器经自己的构造路消除偏离或恢复 Q : 形态上 $\pi(M_t) \neq G_t \rightarrow \pi(M_t) \cong G_t$, 功能上成员合于规格, 能力上可繁殖等性质恢复; R 与 Ω 零改动	被删账本的 channel 按 decl 重建 (形态, E3); 不合规的成员被机器用自己的 syscall 路换掉 (功能, E2-c); 被错误退役、承担繁殖角色的成员按规格重写装回 (能力, E5)	§4.3、§4.5、§4.6
自进化	一次经选择的变化进入 G 并沿谱系遗传 (§3.4.3): 变异与选择的内环在膜内	机器缺零件, 作者写出、编译、装入, 零件进 G , 子代逐字继承 (E1, 真模型); 被退役的成员按规格重写、装入, 两代继承 (E5, 真模型); world 层的变异归规则遗传 (E4, §4.4)	§4.2、§4.5
自复制	$\text{decl} \rightarrow \text{pack} \rightarrow \text{spawn}$ 闭合; $\text{decl}(\text{子}) = \text{decl}(\text{父})$ 且子代能继续繁殖; 继承 G (含 world), 不继承宿主侧的文件与效果	子代逐字继承零件、不继承文件; 三代携带同一段重写的成员, 父代与子代分别用它产生下一代	§4.2、§4.5
自组织	多台机器凭遗传的协议与消息各自生长本地拓扑并维持之; 没有膜外写拓扑, 没有共享形态与共享账本; 协调者是一个普通、可替换的内部成员	三台机器各自长出指向另外两台的门; 早到的消息丢失, 下一次心跳自愈; 被硬杀的机器由邻居照它自己的账本唤醒	§4.3

四行不互斥: E5 同时是自维护、自进化与自复制。五个实验在账本与世代上的位置见图 5。

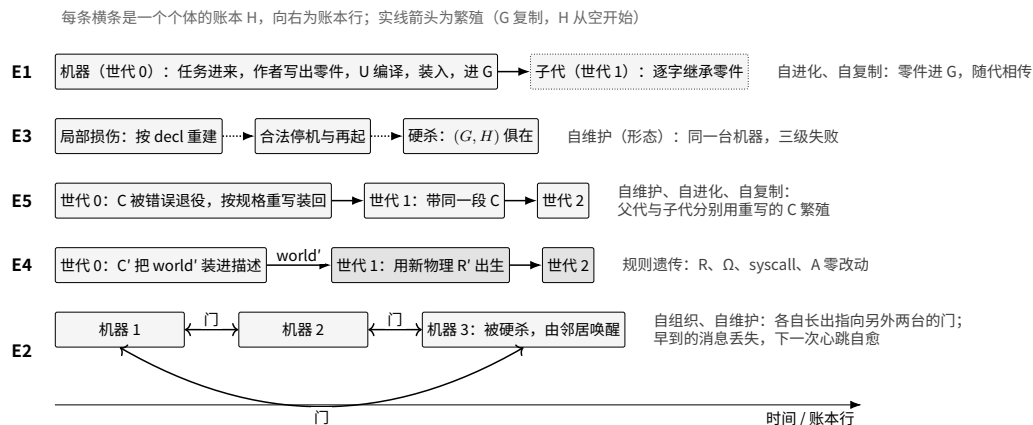


图 5: 五个实验的示意: 每条横条是一个个体的账本, 向右为账本行, 实线箭头为繁殖。E1 一个任务的全轨迹与子代继承; E3 同一台机器上的三级失败; E5 重写的 C 跨两代; E4 换入 world' 的子代用新物理出生并继续繁殖; E2 三台机器凭遗传的协议各自长出出门并在硬杀后由邻居唤醒。

四句都是机制主张。每个实验里”变什么、修什么、要不要”由膜外给定 (任务文本、规格、诊断); 机器交付的是”怎么变、怎么修、变了之后如何遗传”这条路。“机器自己决定变什么”在膜外 (§5.3)。

4.2 一个任务的全轨迹 (E1)

装置是能写出的最小完整机器：{c0, c1, c2}, c2 = {L, U, 两扇门}, L 的远端是 deepseek-chat。机器出生、静止；实验者经门发出唯一输入：

task
把 hello 写进 notes.txt 再读回来

机器里没有 file 这个成员。账本：

seq	账上发生的事
5	task 进账 (门 → L: 第一次调用开始)
6-7	L call("0", "show"), call("0", "who")——读历史、读形态 (§3.4.6 的自省, 入账的事实行)
8→10	L 把候选源码发给 U: U test v1 → result 1 (头行解析错)
11→13	v2 → result 1 (漏了测试分隔符, 测试没被执行)
14→16	v3 → result 0 (通过)
17	L 经门请 c0: add c2 program tag=file iface=...——14 行源码随行入账
19	第一次调用收笔; step 帧 [0, 0, U, U, U, c0], 无返回值
20-21	R 放入并实例化 c2/5; placed 回执经门回来——第二次调用开始
24-29	L call("file", "write notes.txt\nhello") → written; call("file", "read notes.txt") → hello——调用的是装进机器的零件, 不是 U
30	done 经门送达发起者

全程约九十秒, 两次调用共约六轮 HTTP。全部机制各动了一次: 任务经门入账 (§3.2.5); 作者组装自省 (§3.2.2); 候选在 U 处两败一胜 (§3.4.3 的内环, 两轮 debug 在账上); 新能力进入机器只有一条路: 门 → c0 → syscall → R 放入 → 登记 → 回执成为下一次调用 (§3.4.1); 机器随后调用自己刚长出的成员完成任务。

这个实验有一个失败前身。第一次运行 (E1-a), 提示语用的是机制测试的措辞, 模型逐字照办: 写了一段把任务硬编码在函数体里的一次性零件, 装而不用, 直接报 done。修复没有碰 R、Ω 与构造器官 c0、c1, 改的是 L 的 text 里那段提示语 (“缺能力就添通用零件, placed 后用它, 路径相对 cwd”), 重跑即得上表。两次运行相差的只是 G 里的一段文本, 行为整个不同: 作者的行为是 G 里的 text (§3.4.3)。

写文件这件事, U 一开始就做得好——它执行任意 Python, E1-a 正是在 U 里把文件写了出来; 机器里什么都没有多。E1-b 做的是同一件事, 多出来的是一个成员: file 有名字与地址, place 行携带它的全部源码落在 H 里, placed 登记进 G, 任何成员可以调用它, 重启后重新实例化, 子代逐字继承。E1 不增加 L 与 Exec 的计算表达力; 它使一项能力构成化——成为机器构成的一部分: 一项潜伏在通用执行器里的能力, 被提升为命名、可寻址、持久、可复用、可遗传的成员 (§3.4.3)。对一段脚本, 两者没有区别; 对由许多部件构成、须长期维护、复制与迁移的系统, 区别是系统存不存在 (§1.1)。

遗传: spawn 之后, 子代的 c2/5 就是 file, text 与父代逐字节相同, decl(子) = decl(父); 子代目录里没有 notes.txt——对子代的 file 发 read notes.txt, 得到 FileNotFoundError; 发 write, 子代自己的文件出现, 父代的不动。零件在 G 里, 文件在宿主侧: 继承的是描述 (含 world), 不是宿主侧的效果——§3.2.3 那条膜在遗传上的对应物。

验收: 自进化 (变化经内环进入 G, 子代逐字继承), 自复制的前半 (逐字继承、不继承宿主侧效果)。边界: 提示语的例子点名了 file——造什么被引导, 怎么造、怎么进来无引导; 单次运行; E1-a 与 E1-b 是“text 即策略”的对照, 不是受控重复。

4.3 种群 (E2): 自组织与自维护

装置: 同一台机器, L 换为 deepseek-v4-pro; 任务文本是两个成员的行为规格——hub (应答报到、广播成员表) 与 reporter (心跳时 ping 同伴、失联则 spawn)——协议四个词, 实现留给作者; 时间按 §3.1.3 从膜外来 (实验者向收件箱发 tick)。

装机 (d0 的作者账本, 从规格到装机八分钟):

seq	账上发生的事
9	task（两成员的行为规格，1160 字）进账（门 me → L）
10–11	L 读 show、who（组装自省）
12→14	L 交 U: run os.path.abspath('.') → result 0: /tmp/t1-real（取运行目录，规格允许）
15→20	两个成员的源码在 U 里各过一遍（result 0、result 0）——体外迭代，无一 err
21	经门 c0: add c3 program in tag=hub——hub 源码随行入账
23	add c4 program in bind=spawn tag=reporter——reporter 源码（约 3000 字）随行入账
25、27	peer c0 c3; peer c0 c4（连线）
29	add c4 door tag=hub → file:/tmp/t1-real#c3——reporter 到 hub 的门
31	done 经门 me 达发起者
33	这次调用收笔: step 帧 [0, 0, U, U, U, c0, c0, c0, c0, me]
34–50	placed 回执陆续经门回来（c3/1、c4/1、c0/6、c3/2、c0/7、c4/2、c4/3），各成一次新调用

随后实验者向收件箱发 tick: hub 回路成立。实验者经门请 c0 生出 d1、d2，两个子代携带遗传的 hub 与 reporter；出生的数量由膜外给定，此后的拓扑由各机器自己长出。自组织从一个子代的视角看——d1 的 reporter 账本，它的成员全部是遗传来的，任务文本从未发给过它：

seq	账上发生的事
1–3	出生时的形态（继承自 G）: reporter（in, bind=spawn）、门 →c0、门 →hub
4	start——第一次调用
7	reporter 经门向 hub 报到: hello file:../spawn/d1#c4
10	hub 广播成员表: peers〈三个端点〉
13、15	对两个陌生端点，各经门请 c0: add c4 door
19、22	两扇新门落账——拓扑长出来了（§3.2.4: 拓扑 = 门的集合）
25、28	对两扇新门各发 ping
31–32	pong 回来——三台互通，距 d1 出生六秒
11、18	插叙: d2 的 ping 早到，此时回指门未就位，署名 door; 20、23 处 reporter 回的 pong 写给 door，被介质丢弃（§3.2.2）——下一轮心跳自愈，丢消息的恢复是协议的一部分

三台之间没有共享的账本、没有跨机的形态，只有各自的器官和门：是生态，不是个体（§5）。协调经 hub：它收集报到、广播成员表，reporter 据此长门。hub 是一个普通成员，装法与 file 同一条路（§4.2），可替换、可退役；拓扑不由膜外写入，由各机器按遗传的协议在本地长出。

然后是损伤。实验者 SIGKILL 掉 d0 之后，d1 连续 tick 却不 spawn。bug 在账上直接可见：

seq (d1/c4)	账上发生的事
94	（修复前）tick 的 step 帧: ... , spawn , ...——裸动词。作者写的是 call(“spawn”, d), 两个参数；此介质的动词参数必须与动词同在头行（附录 A），于是头行只剩 spawn，解析不到成员，被静默丢弃：失联检测判真，救援从未发出，账上无任何回执

一处拼写，整台机器的远端维护失灵：ABI 是真实的变异面。修复走机器自己的路 (E2-c)：诊断在膜外完成，经门递入 d1 自己的 c2 (附旧源码与定位)，约 70 秒后修复落账。两本账各记一半：

seq	账上发生的事
c2:25 c2:28	修复任务进账 (附旧源码与 bug 定位) L 写出新 reporter, 经门 add c4 program in bind=spawn tag=reporter——源码中可指认 call("spawn" + d) placed c4/6 回执
c2:31 c2:34→37	retire c4/1 → retired——§3.4.2 的" 没有 replace, add 新 + retire 旧" 作用于活着的成员 新 reporter 落座 (in: 接任接待员); 旧件退役 tick——新件的第一次心跳
c4:111-112 c4:133 c4:138	spawn 回执: /tmp/t1-real pid=2705088——d0 照它自己的账本醒来 (§3.3.2 的折叠, 由邻居代按 开关)
c4:146	这次调用的 step 帧: ... , spawn /tmp/t1-real , ... ——与 seq 94 一字之差
c4:147	pong 回来, 回路恢复

验收：自组织（互长门、心跳自愈、硬杀的机器被邻居照其账本唤醒——维持归种群），自维护的功能一档（不合格的成员被机器自己的 syscall 路换掉）。边界：任务是照规格实现，考实现与接线，不考发明；bug 的诊断由实验者完成、经门递入，选择与评价在膜外；单次运行。另有一处驱动竞态最初与被试失败混同，靠账本区分。⁵

4.4 世界 (E4)：换掉 R 的子代仍能繁殖

§3.3.5 说 world 对个体冻结、对谱系可变，由 E4 (机制层，桩作者，确定性) 见证。变异从承担打包的成员进入： $C' = \text{原 } C + \text{一处幂等补丁}$ ——pack 前把 G.world 里 R 的记账函数改为每行多盖一个常量戳 "w": 2。消息原文照录，R、 Ω 、syscall、A 零改动：

步	介质里发生的事	断言 (对账本与磁盘)
1	经门: retire c0/C——C 不是接待员, 退得掉	retire 行落账
2	经门: add c0 program tag=C bind=syscall,spawn + C' 源码——同角色接替, 占回同一逻辑地址	旧件 retired、新件 tag=C
3	经门: spawn w2——普通地生, §3.4.4 的六步无一步特殊, 走的已是 C'	spawn 回执带 pid
4	—	父代全部账行无 "w" 键: 个体不可改
5	—	子代目录里 runtime.py 含 "w": 2 (world 是 R'); 子代 c0、c1 账本每一行带 "w": 2 (每行都是 R' 写的)
6	—	子代的 C 与 C' 逐字相同 (地址重排, 角色不变); decl(子).world 含标记——变异进了登记处, 可遗传
7	向子代发 spawn w2g——不动点: 在 R' 上繁殖仍然成立	孙代出生、发育, 回执带 pid

⁵E2 的原始账本运行于生命周期语义收紧之前的实现 (重启事件逐 channel 注入; 现仅在首 channel 记录, §3.3.1)。原始记录不改写; 本节结论不依赖物理事件落在哪本账, 当前语义见 §3.3.1 第 5-7 条与 §4.6。

步	介质里发生的事	断言（对账本与磁盘）
8	—	孙代 runtime.py 含标记、账本每行 "w": 2; 孙代的 C 仍与 C' 逐字相同（补丁幂等，空转不叠加）

同一份账本见证 §3.5 所述规则遗传的两种节奏：成员 C' 在父代代内装入（第 2 步），随即用于繁殖（第 3 步）；world' 由 C' 写进子代描述，只在子代生效（第 4、5 步）。

变异选在记账规则上，理由有三：可观测性最大，任抽一行即可判定跑在哪个 R 上、有没有被波及（第 4、5 步）；无害性最大，不改任何转移规则，R 与 R' 的机器互相生养，实验测的是这条路通不通，不测新 R 对不对；幂等可判（第 8 步）。

验收：规则遗传 (§1.2 第四条)——world 变异经普通成员的装入路径可达，个体冻结，隔代生效，遗传闭包成立；自进化的内环未经此路。边界：变异内容是实验者规定的，C' 由实验者写成、经门装入，不经 L 与 U；标记不改任何转移规则。规则遗传的路径已通；D 沿此路写出 world' 的端到端运行未做。

4.5 成员 (E5)：丢掉的 C 被重新写回来，两代用它繁殖

§3.5 把完整性归为器官维护的性质：丢一个成员，机器不是死了，是暂时不完整。实验取 C：c0 里承担繁殖角色的成员，也是三个角色中能经合法转移丢掉的那个——A 是 c0 的接待员，R 拒绝退役当前接待员；B 是 C 流程中的 pack。

本次运行中作者未取回旧源码：任务未附旧源码，c2 的账本上只有对本 channel 的 show 与 who；旧 C 的 text 仍在 c0 的 H 中，取回需 c0 的接待员经门送出，当前 A 不提供这条接口。写回的 C 与原版结构不同（下文其二）、满足同一协议、撑起两代繁殖——重新创作是行为上的结论。

真模型版 (E5-b, deepseek-v4-pro)：任务只给协议规格——C 必须实现的两步应答——不给旧源码。全程四分钟，三本账各记一段。

丢失与重写（父代 c0 与 c2 的账本）：

seq	账上发生的事
c0:47–48	实验者经门：retire c0/C——承担繁殖角色的成员退役；retire 行落账
c0:54–55	spawn kid：A 把请求转给 C（step 帧 [C]），而 C 已退役——无回执、无目录，繁殖能力确已丢失
c2:9	修复任务（协议规格，1451 字，无旧源码）进账
c2:10–11	L 读 show、who（组装自省）
c2:12→14	候选交 U：run（2084 字）→ result 0
c2:15	经门：add c0 program bind=syscall,spawn,stop tag=C——2112 字的 C 随行入账
c2:17	这次调用收笔：step 帧 [0, 0, U, c0]
c0:58	place tag=C, textlen=2112, by=1——机器自己的手装回了成员
c2:18–21	placed c0/C 回执到来（新调用）→ done 经门达发起者

繁殖恢复（仍在父代 c0 账上）：

seq	账上发生的事
66–67	再发 spawn kid：A 转给 C，这次 C 应答
69→77	C 问 decl；c1 折账交出 G（48147 字）
79	spawn 回执：spawn/kid pid=2873033——子代进程起来，根门开着

seq	账上发生的事
85→99	C 请 A build: 六条 syscall 经门逐条发进子代根门——create c0、add A、门 c1、门 c2、add C (连同它自己, 2218 字)、出生证明门 (§3.4.4 第 4 步)
100	C 经根门发 msg c0 start + G (74815 字) ——第一条消息, 关门切离
102	spawned /tmp/restore-real/spawn/kid door=kid

遗传 (子代与孙代的 c0 账本):

seq	账上发生的事
kid:1-5	经根门放的五行 (by=_root): A、门 c1、门 c2、C: textlen=2112——逐字继承、出生证明门
kid:6→35	start 携 G 进账; 子代自己的手 (by=1) 长出 c1、c2 与全部连线——发育完毕 (§3.4.4 第 6 步)
kid:38、43 kid:70、93	spawn grand 进账; A 把它转给继承来的那段 C 回执 spawn/grand pid=2877373; spawned .../kid/spawn/grand——谱系写进了路径 (§3.4.6)
grand:4	place tag=C, textlen=2112, by=_root——第三代, 同一段 2112 字
grand:6→35	同一套发育序列原样再走一遍

三代携带同一段作者写的 C; 父代与子代分别用它产生了下一代, 孙代继承并完成发育——与 §4.4 的不动点同一个验收形状: 新写的东西自己撑起下一次繁殖, 才算补上了。

三点记下。其一, 无特权的作者造出了有特权的成员: L 与 U 都 $\text{bind} = []$ (§3.4.3), 它请求的 add 带 $\text{bind}=\text{syscall}, \text{spawn}, \text{stop}$, c0 照办——特权由 c0 授予; 是否审查这类请求是 A 的 policy, 不是 R 的事。其二, 写回的 C 与原版不同但等价: 它用 who 找接待员 (原版用启发式)、按回执差集取未办请求 (原版按计数)、处理了原版没处理的嵌套路径——协议对了, 实现是它自己的, 即重新创作而非恢复备份。其三, R 那条”不能退役当前接待员”的结构规则保住了三个 channel 的接待员位置——A、登记员、L 都是接待员, 不能被直接退役; 能直接丢的核心成员是 C 和 U。R 保护的是接待员的位置不被直接清空, 不保证接任者可用 (替换路径照走); 能力的完整性归器官 (§3.5)。

验收: 自复制的后半 (三代携带, 两代繁殖), 自进化 (重写的 C 经内环进入 G, 两代继承), 自维护的能力一档 (被退役的繁殖成员按规格重新创作补回, 可繁殖性恢复)。边界: 规格是给的, “机器自己发现缺了什么、决定要补”在膜外; 单次运行。

4.6 崩溃、停机与损伤 (E3)

单个成员的失败是最轻的一级: 调用抛异常, R 在它的 step 行记 err、回复为空, 游标照推; L 的远端端点不通, 同样是一行 err, 机器活着——成员的失败是机器历史里的一行, 不是机器的死。§4.2 里候选源码的两次 result 1 不在这一级: 那是 U 在自己的检验里捕获的失败, 作为正常回复返回, U 的 step 没有 err, 失败只存在于内环的选择结果中 (§3.4.3)。

重一级是事件中途的崩溃。§3.3.4 已立: 未落账的一切不属于机器; 已落账的部分效应是历史——syscall 在调用中当场执行并落账, 外层的 step 行要等调用结束才写, 这个窗口里崩溃, 机器醒来时处在形态被改了一半的状态。这是 §3.5 所述的损伤态: 账本仍然真实, 把 $\pi(M_t)$ 拉回 G_t 是自维护的操作性定义; R 不需要事务, 也不需要两阶段提交。

重放不能替代再投递: channel.add.actor 不幂等, 重放一条 add 会产出第二个成员 (重名由 R 分配新 tag), 实际形态里有两个、G 里记一个, $\pi(M_t) \cong G_t$ 不再成立; 多出来的成员是损伤, 拉回 G_t 归器官。§3.3.4“再投递, 不回放”的一半理由在此; 另一半在 §4.7。

以下三段轨迹由机制实验给出 (E3, 桩作者, 确定性); 断言对账本与磁盘。

局部损伤与重建。装置: 最小机器加一个成员 x (回声程序); 损伤在宿主侧。

步	介质里发生的事	断言
1	经门: add c8 program in tag=e; peer c0 c8	c8 账本有 place 行; c1 有 placed c8 登记; 记下 decl
2	停机; 宿主侧删除 c8 的账本文件	—
3	同一个 P 被 Ω .spawn 再起, R 折叠	M_t 里没有 c8 (channel 存在 $\Leftrightarrow$ 账本至少一条 place 行); 首 channel 多一条 up
4	A 收到 up, 向 c1 发 reconcile, 对照 decl 逐 channel 发 syscall	未损的 channel 回执 exists、跳过; c8 被重造: 同一段 text、同一 tag、接待员位、指回 c0 的门
5	—	c8 的账本从空开始: 照描述重造, 不回放旧账; c1 不重复登记 placed c8; c1 账上无 up、down
6	—	——物理事件只落首 channel decl 与第 1 步相同: $\pi(M_t) \cong G_t$ 恢复

修复由 A 的 text 完成——把 up 翻译成 reconcile、对账、重建是 A 的策略; R 只做折叠与 up。这是自维护形态一档的见证: $Q = G_t$ (§3.4.2)。

合法停机与再起。装置同上, 机器以独立进程运行。

步	介质里发生的事	断言
1	出生、发育; 向 x 发 hi	x 账上 hi 与它的 step
2	经门 c0: stop—— $A \rightarrow C \rightarrow$ stop 动词	c0 账上回执 stopping; 首 channel 记 down, 为最后一行; 进程退出; x 的账上无 down
3	停机期间向 x 发 late	躺在收件箱
4	同一个 P 被 Ω .spawn 再起	首 channel 生命周期行为 down、up; late 入账并被处理
5	再次经门 stop	生命周期行为 down、up、down; x 的 place 行与 decl 与停机前逐行相同; 游标跨进程不丢

硬杀。装置同上。

步	介质里发生的事	断言
1	向 x 发 late, 消息落账、尚未被跑时杀掉 R	—
2	再起	生命周期行只有 up, 无配对的 down——据此判定上次为异常终止; 游标由折叠恢复; late 在待办中, 被重跑 (至少一次); decl 不变
3	账上堆着 a、b 未办, 经门 stop	直接放弃: a、b 不做; down 为最后一行
4	再起	a、b 重跑, 回声 echo:a、echo:b

外部 SIGTERM 与 SIGKILL 同属这一类: 都不写 down。停着的机器不会自己醒: 唤醒来自 Ω .spawn——实验者、人, 或另一台机器的 spawn 动词 (§4.3)。E2 的机器带着无 down 的记录醒来, pending 的请求重跑: 至少一次, 语义如 §3.3.1 第 6、7 条。

4.7 活性边界

§3.5 说修复器官也损坏时，从损伤回到完整的路可以不存在。这条边界在此画出。器官维护完整性，成立的最小条件有四：(i) 机器跑得起来；(ii) c0 的接待员活着，或已按升级纪律备好后继；(iii) 有作者可用（膜内的 c2，或经门的膜外作者）；(iv) 目标成员的描述、协议规格或等价的选择判据仍可。四条齐备时，丢失的成员能经构造路重建；§4.5 对 C 给出直接见证，恢复所依据的正是实验者递入的协议规格。缺第四条时，作者在场也不知道该写出什么。前两条各对应一条活性失败。

手够不着。所有形态改动都经 c0 的接待员；它一死，请求进不来，新接待员也装不上——装它恰需要经过它。活着时它随时可以换掉自己（先加新的、指过去、再退旧的；E2-c 里接待员接任真实发生过），不可恢复的只有死于无后继的那一刻。

身体醒不来。设一条消息能在处理中杀死 R——诱发宿主资源耗尽、踩中绑定实现的洞，来源不重要。机器崩溃；重启 = 折叠账本；那条消息还在账上、还未办完；再投递，再崩。凡重投同一条未完成事件而不隔离它的恢复机制都在此循环：一个坏点冻住整台机器，历史里的毒随历史永生。这是 §3.3.4 不选全量回放的另一半理由。R 保证的只有账本真实、折叠可恢复 (§3.3.2)；识别毒消息、跳过、隔离、退避，归工程。毒消息比成员失败深一级：它杀的不是某个成员，是物理本身；邻居的唤醒也救不了这一种——判死 → spawn → 再投递 → 再崩，§4.3 那条救援路在此失效。

两条边界杀死的都是个体。个体停在一个定义良好而不可恢复的状态 (§3.4.2)——毒消息的情形里转移仍在发生，唤醒与崩溃循环，回到完整的路径不存在——带着身份与历史；G 不随个体死去——它在自己的登记处，在每个子代的包里，在每台亲缘机器的 decl 里。谱系有幸存者时，重造一台是一次普通的出生 (§3.4.4)；§4.5 给出更强的一档：G 缺了一块，作者能照规格把它重新创作出来。个体的死是真实的，H 断了就是断了；谱系与个体的账在 §5.3。

5 讨论

§2–§4 讲的是一台机器：照什么原型，由什么构成，跑起来如何。对 agent 而言，机器不是目的，三个问题才是：这台机器与 agent 是什么关系 (§5.1)；agent 放进机器，多了什么 (§5.2)；为此付出什么，机器又不承诺什么 (§5.3)。答案指向 §3 的定义与 §4 的记录。

5.1 Agent 与机器的关系

Agent 今天的构型是模型加外围。模型负责生成；memory、tools、workflow、subagent、gateway 与验证器围在模型外面 (§1.1)。生命周期全在外围手里：装入靠加载器，重启靠进程管理，复制靠镜像，升级靠部署脚本，身份靠部署位置——同一份模型与提示语部署两处，是两个 agent 还是一个，由部署者裁决。模型在运行中写出的新组件能否成为 agent 的一部分，同样由这些外围裁决。这个构型里“agent”一词的指代范围没有定义：模型的 loop、tools、skills、配置、部署环境，哪些属于 agent、哪些不属于，由框架与部署者各自决定，随部署改变。于是“agent 管理和修改自己”这句话无法判定——被管理、被修改的东西算不算 agent，取决于界线画在哪里，而界线没有统一的契约。

Dalek 的构型是机器加成员。散落在外围的构成机制收进机器，剩余的依赖列入 Ω 或作为输入进入；生命周期的每一步是机器自己的转移：触发与材料可以来自界外，但一次变化不算装入、升级、复制或延续，变化之后机器是什么，历史如何延续，什么进入遗传，相对于 Ω 由 G、R 与 H 定义。模型是机器里的一个成员 L：c2 里的一个程序 actor，bind 为空，可替换，可并存两个，它的 loop 是 G 里的一段 text (§3.4.3)。两种构型逐项对照：

	模型加外围	机器加成员
装入一个新组件	加载器读文件	add 落账，placed 登记进 G (§3.4.1、§3.4.2)
重启	进程管理器重起，状态看外围存了什么	Ω .spawn 起 R；R 折叠 H、重建形态， Σ 归零 (§3.3.2)
复制	镜像与部署脚本	decl → pack → spawn (§3.4.5)
升级	换镜像、改配置	add 新成员、retire 旧成员；换物理生一个子代 (§3.4.2、§3.3.5)
身份	部署位置或外部命名	(G, H) (§3.4.6)
模型写出的新组件归谁	文件系统、加载器与操作者	经 c0 成为成员，随 G 遗传 (§4.2)
模型的位置	生成中心，通常被称作 agent	一个成员，作者
谁在维护	外围的人与脚本	修复路径与授权归机器的器官；目标、诊断与触发可来自界外 (§4.3、§4.5)

左列每一项由一个各自约定的外围裁决；右列每一项是 §3 的一条转移，触发可来自界外，转移的含义由机器定义。§1.4 所说的组织中心从任务循环移到生命周期，展开即此表：任务循环仍在——L 的 loop 就是它——只是它从组织中心变成了一个普通成员。

四个自主语由此确定，且落在三个尺度上。个体：自维护维护的是这台机器的形态、功能与能力对目标 Q 的关系。世代：自复制是父代对子代的关系，复制的是 G；自进化是谱系的变化——候选的生成、可运行性的检验与装入在个体内发生，变化进入 G 并被后代继承才算完成 (§3.4.3)。种群：自组织组织的是多台机器之间门的拓扑 (§4.1、§4.3)。三个尺度用同一套构造，主语没有一个是 L。L 在每一处的位置是作者：写出候选，请 c0 装入，与膜外的请求者走同一条路 (§3.4.3)。模型加外围的构型写不出“agent 管理和修改自己”这个句子，因为句子的那个“自己”没有主语：修改者是模型，被修改的是模型外的组件，维护与裁决的是界外。本文的工作可以概括为给这个句子造一个主语。主语造出来之后，这句话有确定的操作含义：一台机器里的成员经 call 请 c0 改了这台机器的 M_t 与 G_t，改动由这台机器的 H 见证，进入 G 的部分沿谱系遗传，H 留在这台机器；维护它的是同一台机器里的器官 (§4.3、§4.5)。

“智能的单元”取计量读法。它不说这台机器聪明，说的是智能以它为单位被个体化、被维持、被遗传：L 提供智能，机器提供单位。介质对 L 的内部实现无感知，L 只经普通的 actor 与 message 接口呈现；机器对 L 的原生性质逐项适配 (§2.2)——为它而造，不以它为中心。

5.2 机器增加了什么

agent 放进机器之后多了什么，分两条轴回答：一条是能力，一条是构成。

能力轴指计算表达力：能算什么由 Exec 给定，能写出什么由 L 给定，D 不增加二者之一；在固定的 Ω 与 L 下，装入新成员不改变机器在这条轴上的位置 (§3.4.3)。E1 里 U 一开始就能写文件，E1-a 正是在 U 里把文件写了出来，那一次机器里什么都没有多 (§4.2)。器官层没有任何东西能做 U 做不到的计算。

构成轴上是机器的全部。E1 多出来的是一个成员：有名字与地址 (§3.2.2)，完整 text 落在 H 里、登记进 G_t (§3.4.2)，任何成员可以调用它，重启后重新实例化 (§3.3.2)，繁殖时逐字继承 (§3.4.5)，换代时 add 新、retire 旧 (§3.4.2)。机器的能力增长即成员在这条轴上的积累。

构成不指部件之间的接线。清单式系统也在接线，动态生成的组件也可以被调用；区别是 §1.2 的四项义务：一项能力成为有构造性定义的机器的成员，它何时成为机器的一部分、以什么形式持久、由谁批准，由机器自己的构造语言与合法转移回答，产生合法后继的规则随后继遗传。对一段脚本，两条轴的差别为零；对由许多部件构成、须长期维护、复制、迁移与升级的系统，差别是系统存不存在——“做到过”是能力轴上的一次事件；“总是如此”——成员资格、持久形式、合法转移与遗传后果始终有定义——是构成轴上的性质 (§1.1)。

机器复杂度的增长落在 G 里，不落在 R 里。R 只认识地址、kind 与 text 作为参数 (§3.3.3)；机器从三个 channel 长到任意多的成员与器官，不给 R 增加任何专用分支。这是万能构造器的结构 (§2.1)：待造物越复杂，长的是描述，不是构造器。关于 R 另有三件独立的事实：G 全部改名后 R 的行为不变，R 对组织名盲 (§3.3.3)；R 是 G.world 里的一段文本，跨代可表示、可替换 (§3.3.5)；替换之后繁殖的不动点仍成立 (§4.4)。

genotype 与 phenotype 在此有严格含义。G_t 是登记处折出的遗传形态，M_t 是介质折出的实际形态 (§3.4.2)。两种比较各答一问。共时比较 $\pi(M_t)$ 与 G_t：同构是健康， $\pi(M_t)$ 偏离 G_t 是损伤。历时比较 G_{t+1} 与 G_t：不同即变异，无论它由新成员的登记还是由登记的写歪产生；变异被后代继承且仍满足验收，是通过了选择的变异。一次变异登记完成后，新的 M_t 与新的 G_t 重新同构。

5.3 代价与边界

机器有了构造性定义，它的代价与能力边界也就可以准确写出。

代价来自介质的同一组决定：账本只追加，形态从描述重建，历史不随描述走。

1. 没有覆写。place 行不可改写，物理地址不复用；改一个成员 = add 新的
 - retire 旧的，新件拿新的物理地址，可以接任同一个逻辑 tag (§3.4.2、§4.4)；接待员的更替同样是 add 新件带接待标记、retire 旧件 (§4.3)；world 对个体冻结，改物理只能生一个子代 (§3.3.5)。模型拒绝的是覆写旧成员与旧历史，不是同一逻辑地址上的在线接替。
2. Σ 不过重启。活着的机器是 (G, H, Σ)，个体是 (G, H)；重启重建形态，不重建现场，要活过重启的成员自己从 H 里重建 (§3.3.2)。
3. H 不随繁殖继承。子代以空的 H 开始；父代的经验只以进入 G 的形态传下去，不以历史传下去 (§3.4.6)。

4. **个体会死**。接待员死于无后继，或一条毒消息把唤醒与崩溃锁成循环，个体停在定义良好而不可恢复的状态 (§4.7)。谱系不死不等于个体不死：H 断了就是断了；谱系的出路是重造一台，那是一次普通的出生，不是这一台的恢复。

能力边界来自同一条：机器交付的是构成路径及其持久、恢复与遗传的语义，不要求意图内生；本文的实验中，变什么、要什么、算不算好，由膜外给出。

1. **四个自是机制主张**。每个实验里变什么、修什么、要不要，由膜外给出——任务文本、规格、诊断；机器交付的是怎么变、怎么修、变了之后如何遗传 (§4.1)。“自”指构成机制及其转移语义属于机器，不指意图的自主，也不指脱离 Ω 的物理自足。
2. **规格在本文中是隐含的**。G 只承载描述；机器应具备什么能力及其验收条件，存在于任务文本与实验者的判断中，经门递入 (§3.5 脚注)。自维护因此是对已固定目标 Q 的命题，目标的自主生成不在本文之内 (§3.4.2)。
3. **真模型实验全部是单次运行**。主张的是这条路存在且每一步在账上，不是成功率 (§4)。
4. **膜是组织与账本的膜**。成员在运行内部对文件与网络的读写不入账、不遗传，主张限于构成性作用 (§1.3、§3.2.3)。

账本是检验这些主张的仪器，理由在主张本身：机器的定义是一切构成性作用经 call 落账 (§3.3.4)，于是“机器做到某事”指一次构成性作用，它发生当且仅当账上有对应的行；某项性质是否被见证，由 §4.1 的验收句在账本的一段轨迹上判定。账本检验不到的——运行内部、宿主侧效果、意图——恰是上面放弃的。仪器的范围与主张的范围重合，是本文以账本而不以自述或演示作证的原因。

6 相关工作

本文构造的对象落在几组通常分开的工作上。这些工作分别把 agent 取作一次任务循环、一个被托管的进程、一个改进过程、一份运行配置、一台以 L 为处理器的计算机，或一个模拟自身的模型；本文关心的是能够同时承担行动、构成变化、历史与后继关系的生命周期主体。本章按这几组工作陈述，最后说明它们与本文的关系。

6.1 Agent harness

Harness 是围绕基础模型的系统：它编排执行，决定模型如何思考与计划、如何调用工具与行动、如何感知与管理上下文、把产物存在哪、如何评估结果 [Weng, 2026]。OpenAI 把 harness 写成为模型设计的环境——仓库内文档、linter 与 CI 结构测试，规则由 CI 机械执行 [Lopopolo, 2026]；Anthropic 把它写成 planner、generator 与 evaluator 三个 agent 的结构，evaluator 经 Playwright 操作页面打分，一次自主运行长达数小时 [Rajasekaran, 2026]。DeepSeek Harness 把包括 agent loop 在内的每个部件做成插件，会话是只追加的事件日志，模型可见的一切由日志派生；模型在运行中定义的动态包只存在于进程内存，重启即清除，跨重启的装入走配置路径 [DeepSeek-AI, 2026]。

Harness 本身也成为可修改的对象。Meta-Harness 把 harness 定义为决定存什么、取什么、给模型看什么的代码，由一个外层的 coding agent 搜索它，每个候选是文件系统中一个含源码、分数与轨迹的目录 [Lee et al., 2026]；AHE 给每类组件一个文件表示，每次编辑附一条可证伪的预测，运行目录、验证器与模型配置只读 [Lin et al., 2026b]。Weng 主张评估器与权限控制应坐在演化循环之外 [Weng, 2026]。改进循环本身见 §6.3。

6.2 系统

§3 对机器状态与合法转移的表述采用操作语义的形式：配置，与归纳定义的转移关系 [Plotkin, 1981]。THE 把系统排成六层，顺序进程之间的协作由显式的同步语句规定，逻辑正确性由此可以预先证明、实现可以穷举测试 [Dijkstra, 1968]；Nucleus 把策略留在核之外，核只提供进程的创建、通信与控制，操作系统与普通程序之别只在辖权 [Brinch Hansen, 1970]。UNIX 的作者记下，系统几乎从一开始就能够并且确实维护了自己 [Ritchie and Thompson, 1978]。actor 模型把一切成员与作用统一为 actor 与消息 [Hewitt et al., 1973]；Agha 给出 actor 的行为：收到一条消息，发出有限条消息、创建有限个 actor、指定下一个行为 [Agha, 1985]；Lampert 在不使用物理时钟的情况下，以事件间的 happened-before 关系定义因果偏序 [Lampert, 1978]。

当代的 agent 运行时把同一组问题放到 agent 上。Agent libOS 给 agent 一个进程：进程身份、对象内存、消息队列、工具表、经系统调用中介的即时工具、子进程、capability、检查点与持久恢复，并维持一条不变量：模型可见的动作面可以演化，资源权限与允许的信息流不因此隐式扩大 [Zhang, 2026]。Shepherd 把执行轨迹做成一等对象：每个动作是结构化事件，轨迹可检视、可 fork、可回放、可回滚 [Yu et al., 2026]。Cordis 以动态组合演算处理组件的装载与卸载：每个效果连同其逆一起交给运行时，依赖以声明驱动组件的激活与停用；其实现以热模块替换在进程内完成替换而不重启 [Shi et al., 2026]。

Agent OS 把生命周期机制放在托管 agent 的运行时中；本文把介质 R 与组织器官写进 G，使其成为机器自身可遗传的构成。

6.3 递归自改进

递归自改进的工作围绕一个循环：改什么，谁评，怎么保留。STOP 给出骨架：一个改进器按效用函数改进输入的程序，再把自己作为输入运行；模型本身不被改动，GPT-4 上均值随迭代上升，较弱的模型上下下降 [Zelikman et al., 2024]。被优化的对象从提示语、结构化上下文、workflow、harness 代码递进到优化器代码 [Weng, 2026]。DGM 让 agent 改自己的代码库产生子代，档案按性能与子代数选父，每个改动经编码基准验证 [Zhang et al., 2025]；Hyperagents 把 meta-agent 与 task agent 合成一段可编辑程序，修改过程本身可被修改 [Zhang et al., 2026a]。每上一级，“谁在改谁”随之变化；Metaⁿ 固定元操作，递归只作用于它的输入，深度由收敛给出 [Kim et al., 2026]。测量上，在匹配的反馈与推理预算下，自动 harness 演化对简单的 test-time scaling 优势不稳定，held-out 泛化有限 [Wang et al., 2026]。本文不提出新的改进策略；它为这类改进循环补上候选装入、事实见证、后继形成与跨代遗传的构成语义。

6.4 Agent 与冯诺依曼

Agent 研究从冯诺依曼借用过两样东西。一样是 1945 年的存储程序计算机：运算、控制、存储与输入输出分工，指令与数值同存于存储器 M、由控制单元逐条取出并使之执行 [von Neumann, 1945]，agent 由此被写成一台以 L 为处理器的计算机。另一样是 1948 年的自复制自动机：一份描述，与解释它的构造器、复印它的复印器分开 [von Neumann, 1951]；近期工作借此追问 L 能否模拟自身，并把它作为递归改进的门槛。

前者从名词对照开始，然后逐步把计算机体系结构的机制搬过来。L2MAC 自称存储程序计算机：指令注册表是程序存储，文件存储是数据存储，控制单元定序指令并管理上下文，L 是处理器 [Holt et al., 2024]。Mi 等以感知、认知、记忆、工具、行动五元组对应计算机的器官，据此移植存储层次、DMA 与流水线 [Mi et al., 2025]。Lin 等把这条路走到最远：L 对处理器，KV 缓存对缓存，上下文窗口对主存，agent 框架对操作系统；整个架构分为回答“能算什么”的概率执行面与回答“该算什么”的确定控制面 [Lin et al., 2026a]。

后者的当代工作是 Zhang 等把递归自改进接回 1948 年的自复制自动机：构造器、复印器、控制器与描述带对应为通用模拟器、复制器、监督程序与一份联合描述，由递归定理得到能模拟自身的程序，并以此定义内省阈值 [Zhang et al., 2026b]。他的对象是 L 本身。文中判定 LLM 是没有物理架构的符号机器，物质性的自复制对它无关，1948 年的结构性自描述因而换成模型对自身功能动力学的模拟；一个 LLM 的完整描述是它的全部权重，而没有哪个模型在前向传播中能访问自己的权重，文中给出的几条出路之一是外置的、近似的自模型。脚手架代码是文中四个修改层里的一层，用来归类 STOP 与 DGM 一类系统。

描述与运行中的机器如何相连，反射研究更早给过答案：RLL 要求系统的每个部件都以描述普通知识的同一 formalism 表示，改描述即改系统 [Greiner and Lenat, 1980]，Smith 要求内部嵌有一份关于自身的描述，且描述与系统之间有因果连接 [Smith, 1984]；AERA 从一段种子出发在运行中生长，改写以加与删代替整体重写 [Nivel et al., 2013]；Gödel Machine 把全部初始代码写进公理系统，证明改写提高效率后才执行，问的是何时允许改写 [Schmidhuber, 2003]。

个体性一侧，自创生传统把系统定义为以自己的运作生产自己的部件与边界 [Maturana and Varela, 1980]；McMullin 为自复制自动机谱系列出的未决问题里有身份、自举与实际的复杂度增长 [McMullin, 2000]。

6.5 与本文的关系

本文与上述工作的关系有三种。直接继承的是冯诺依曼的两项构造——1945 年指令成为机器可操作的数据 [von Neumann, 1945]，1948 年描述与构造器、复印器分开 [von Neumann, 1951]——actor 模型的成员与消息 [Hewitt et al., 1973, Agha, 1985]，操作系统把策略留在核之外的原则 [Brinch Hansen, 1970]，以及操作语义的状态—转移表述 [Plotkin, 1981]。两项构造在本文汇合为一句：G 是 1945 年意义上的存储程序，它描述的内容是 1948 年意义上的机器本身——器官、作者与运行时——由 A 解释、由 B 复制。独立收敛的是：Meta-Harness 将候选源码、分数与轨迹落入文件目录，AHE 将可编辑组件表示为文件，并把运行目录、验证器与模型配置置为只读 [Lee et al., 2026, Lin et al., 2026b]；Agent libOS 与 Shepherd 分别把 capability、检查点和可操作的执行轨迹纳入运行时 [Zhang, 2026, Yu et al., 2026]。这些机制与本文的账本 H、作者与验证者的分立、bind 与介质边界分别处理同一类问题——不可重算的运行事实、作者与验证者的分立、动作面与资源权限的分离——但不是本文的来源。对象不同的是：这些工作分别定义一次任务循环、一个被托管的进程、一个改进过程、一份运行配置、一台以 L 为处理器的计算机或一个模拟自身的模型；本文定义的是生命周期主体——以描述与历史个体化，相对于宿主契约有明确的边界，由运行时给出合法的构成转移，产生后继的规则随后继遗传。§3 的构造说明 §1.2 的四项义务在这一个对象上同时成立；§4 的账本见证自维护、自进化、自复制与自组织各有一条构

成路径。本文没有分别发明消息、账本、自描述、动态装入、编译或自复制；语言模型在这台机器里是普通成员，它的替换、维护与遗传与其他成员走同一条路径。

7 结论与未来工作

本文给出了一类 agent machine 的构造性定义，并建立了它的一份运行见证。它的中心不是让一次任务循环变得更强，而是给不断变化的能力一个可以持续存在的主体。相对于显式的宿主契约 Ω ，这个主体以有限的介质定义自己的成员、边界、历史与合法的构成转移：以 G 描述自身，以 H 区分自身——机器在装入、重启与改变中保有可判定的身份，繁殖产生的后继与父代明确区分——并使构成变化沿确定的路径发生。大模型由此不再是外围结构所环绕的组织中心，而是机器中可以替换、维护与遗传的作者。

运行记录为自维护、自进化、自复制与自组织各见证了一条构成路径；四者不是四项附加能力，而是同一生命周期在个体、世代与种群上的四个方向。本文由此建立的是机制的存在性：任务、规格与价值判断仍可来自外界，而一次变化是否以及如何成为机器的一部分、持久、恢复与遗传，已由机器定义。Dalek 不是一个更强的 agent loop，而是这些 loop 得以生长、改变并延续的机器。

7.1 未来工作：可能的扩展方向

本文给出的机器是满足 §1.2 四项判据的基础构造。严格的结构没有限制它的发展空间，反而给每一次发展定下了确定的方式：新的能力、器官与协议以成员的形式写进自我描述，经同一条装入路径进入，记入账本，随描述遗传；运行时不因此改动，复杂度只在描述里有序地增长。在保持这四项判据的前提下，以相同的机制和路径增加新的成员、器官与协议，可以让规格与选择进入机器，让大模型本身进入构造路径，让能力在机器之间传播与重组，让机器组成更大的机器。以下按四个方向展开。

7.1.1 规格引入描述

当前机器能够依照一个给定目标 Q 修复或改造自身，但 Q 来自 G ，既有历史或膜外递入的规格 (§3.4.2、§3.5 脚注)。可以在 G 中增加独立于实现描述的规格部分，并增加持有、解释与比较规格的器官，使实现与规格之间出现明确的满足关系。持有选择判据的成员与持有规格的成员是同一类东西；一次变化值不值得要，能否在机器内判定，取决于规格是否附带可执行的判据。规格仍是准静止的 text，随 G 复印与遗传；规格的提出、修改、检查与采用由普通成员经消息协作，在 H 中留下事实。

加入这一层以后，自维护不再只能相对于一次临时递入的目标陈述。机器可以在改变实现时保留规格，可以把 text 不同而满足同一规格的成员视为等价实现，也可以让契约、验收条件与治理规则随组织持久和遗传。它把“能改变自己”扩展为“能在一个明确规范下改变自己”，并给长期维护、替代实现与可审计治理提供共同对象。

7.1.2 L 和 U 也成为机器

本文的 L 与 U 已是介质中的普通成员，但它们的内部实现仍作为黑盒给定。可以分别用一个保持原消息协议的 channel 或 Space 替换它们： U 展开为种子编译器、编译器源码、构建器、测试器与验证器； L 展开为数据流水线、训练器、推理器与评价回路，训练数据与训练过程是它的描述，权重是构造的产物，如同源码之于进程。展开之后，跨 channel 的往来是经门的请求与回信，与 c_0 向 c_1 取 decl 同一样式 (§4.5)；本 channel 内留一个门面成员管理会话，同步的 call 边界不变。初始编译器、加速器或基础模型仍可停留在 Ω 或外部服务，展开到哪一层由当前机器选择；外层仍向 L 发送 text，向 U 发送候选与测试。这一项与尺度协议 (§7.1.3) 相关。

它带来的新能力是生产机制的自维护与渐进内生性。编译器升级、模型训练、评测与回滚可以像其他器官一样被构造、检查、替换、恢复和遗传；机器不只改变已有能力，还能逐步接管产生能力与判定能力的基础设施。自足由此不再是“永远依赖外部服务”与“一开始重造全部技术栈”之间的二选一。

7.1.3 机器即 actor

可以增加一对对偶的组织协议：封装协议把一个由所在 Space 驱动、经接待员呈现行为的 channel，或一台带有自身 G 、 H 的 Space，经桥接器呈现为外层的一个成员；细化协议则用实现同一消息协议的 channel 或 Space 替换一个成员。协议要说明替换所保留的观测——回复、异步消息、故障、生命周期与资源行为——但不要求内外两层具有相同结构，封装之后再细化也不要求还原原结构。桥接器是普通成员或门，跨边界的作用仍是 message，内部机器仍保有自己的边界、身份与历史。待研究的核心是跨尺度的观测等价与异步的控制流：call 同步返回，门返回空，一台机器作为成员被 call 时如何回复，是这项协议要回答的第一个问题。

其价值是让组织增长不再要求同步发明新的元框架。一个复杂器官可以取得独立的维护与故障边界，一台完整机器也可以成为更高层机器中的一个器官，每一层使用同一组组织原语。由于人和 agent 都经 actor 与 message 进入系统，角色授予、监督、决策与更替也可以成为 channel 中的普通器官，治理本身能够递归地进入被治理的组织。复杂性的增长进入 G ，不迫使 R 随组织层级增长。

7.1.4 分享和传播能力片段

可以在完整 G 之外增加一种可移植的 G 片段。片段可以描述一名 actor、一个 channel，或一组带内部拓扑的 channel，并显式列出 imports、exports、接口版本与 world 要求。配套的 export 与 import 器官负责导出、检查、名称重写、依赖绑定与权限重授；通过以后，片段沿现有路由 c0 构造、在 H 入账并由 c1 登记。协议传播的是准静止的构造描述，不是原实例的 H、易失状态、活端点、bind 或已经产生的宿主侧效果；接收方在自己的边界内重新检查和授权。

新增的能力不只是分发更多代码。复用单位可以是一套编译与测试器官、一条研究或生产流水线，或一组带内部治理关系的 channel；一个谱系获得的组织方案可以被另一谱系接纳并与第三个片段重组。能力由此从一次会话中的技巧变成可检查、可维护、可组合、可遗传的组织。

记号

个体 = (G, H): G 答”我是什么”，H 答”我是哪一个” (§3.4.6)；运行态为 (G, H, Σ), Σ 不入账、重启归零 (§3.3.2)。本文说”一台机器”，指一个 Space：由自身携带的 R 驱动，在任何满足 Ω 的宿主上运行 (§3.2.1)。字母 E 只在 §2 用于 1948 年原型的机器 $E = (A + B + C + D) + G$ ；§4 的 E1-E5 是实验编号，二者互不指代。

角色、器官与介质：

记号	名	一句话
Ω	宿主契约	运行、存储、网络三条能力；落地处提供，不随机器运走 (§3.1)
R	运行时	结构不动点：实例化 actor、投递 message、折叠账本、执行 syscall；对功能与组织无感知；在 G.world 里随谱系遗传 (§3.3)
G	自我描述	机器全部形态的清单（含 world、构造器官与 agent 的 loop）；答”我是什么”；可在运行中被改，随复制遗传 (§3.4.2、§3.4.6)
H	账本	只追加、介质盖章的完整运行历史；答”我是哪一个”；不随繁殖继承 (§3.2.3、§3.4.6)
A	构造器	唯一解释 G 的地方：realize (§3.4.1)
B	复印器	只抄不读：pack (§3.4.1、§3.4.5)
C	控制器	编排 decl $\rightarrow$ pack $\rightarrow$ spawn $\rightarrow$ build $\rightarrow$ start (§3.4.4)
D	通用能力制造器	$= L + U$ ；制造对象及于成员、loop 与子代的 R (§3.4.3)
L	大模型（作者）	生成能力候选；普通成员，无特权、可替换 (§2.2、§3.4.3)
U	编译器	把候选 text 变成可装入机器的部件；在作者之外 (§3.4.3)
c0	构造器官	承担 A、B、C；全机唯一解释 G 的 channel (§3.4.1)
c1	登记处	G 的居所；decl = $G_0 \oplus placed \ominus retired$ (§3.4.2)
c2	作者器官	承担 $D = L + U$ ；bind = [], 改形态与膜外走同一条路 (§3.4.3)

形态、目标与派生量：

记号	名	一句话
G _t	遗传形态	c1 对自己账本的折叠：这台机器认为应当遗传与再生的结构 (§3.4.2)
M _t	实际形态	R 对全部账本的折叠：此刻活着的结构 (§3.4.2)
π	出处投影	保留构造器放的一切与经根门放的 c0 成员，去掉出生证明、临时门、退役成员；静止时 $\pi(M_t) \cong G_t$ ，是 c0 维护的性质 (§3.4.2)
Q	维护目标	修复开始前已固定的目标：形态上 $Q = G_t$ ，功能上是成员的规格，能力上是可运行、可繁殖等性质；由 G、既有历史或膜外规格给出 (§3.4.2、§4.1)
world	物理	G 的根字段： ω -bind、loader、R 三样；对个体冻结，对谱系可变 (§3.3.5)
Σ	易失状态	成员的内部状态；不入账，重启归零 (§3.3.2)
P	打包形态	$P = \text{pack}(G)$ ：world 三文件加 G.json；死的，可跨宿主、可比较 (§3.4.1、§3.4.5、A.6)
decl	取描述	c1 交出 G _t 快照的操作；快照在 pack、spawn 与 start 期间不变 (§3.4.2、§3.4.4)
realize / pack	构造 / 复印	realize 读结构、搬 text (A)；pack 抄整份 G、不读 (B) (§3.4.1)

词汇：actor / message / channel / Space——基础构造单元与三层膜 (§3.2.1)；成员——注册在某 channel 里的 actor；门与根门 (§3.2.4、§3.4.4)；syscall——写形态的三条 call：channel.create、channel.add.actor、channel.retire.actor，与动世界的 spawn、stop (§3.2.2、A.2)；膜外——机器边界之外的一切。

A 转移表与 ABI

A.1 状态与事件

状态：每 channel 一本只追加账本 + 每 actor 一个游标。事件：某本账上多出一条写给某地址的消息 (不带 run 标记的 msg 行)。一条事件 = 调用该成员一次，run-to-completion；嵌套 call 构成调用栈，只有事件推游标。

A.2 完整转移表

kind	实例化 (place 折叠时一次)	每条消息
程序	Exec.load(text, {call, me, channel})——源码 exec 一次，须定义 run(m)；m = {seq, from, to, body, channel} — (text = 端点地址)	run(m)；返回值 = 回复 (None/空 = 无)
门		Port.send 原样送出，介质署名本 channel 端点；返回空
放 actor (syscall channel.add.actor)	R 在活跃路由表分配唯一 tag (重名加数字后缀)，写 place 行，完整 text 入账	回执 channel/tag

另两条 syscall: channel.create (建 channel)、channel.retire.actor (退役 ——追加 retire 行, 地址不复用, 当前接待员不可退役)。世界动词: spawn (起子代进程)、stop (合法关闭协议)。

A.3 call 的地址空间

地址	语义	需要绑定
tag	本 channel 成员; 名字在调用那一刻解析	否
0	读介质: show [a] [b] → 账本行; who → 此刻成员表。读不是能力, 全员开放; 账上只记事实行	否
门的 tag	送出膜外, 返回空	否
channel.create / channel.add.actor / channel.retire.actor	写形态	syscall
spawn / stop	动世界	同名

写给不存在的地址: 丢弃。写给成员: 同步调用, 对方返回值即 call 返回值。

A.4 账本行

每 channel 一个 JSONL 文件, 单写者 (本机的 R):

k	关键字段	谁写
place	seq, addr, kind, text, bind, in, by, tag, iface?	R 执行 add 时; 带完整 text
retire msg	seq, addr seq, from, to, body, run?, at?, by?	R 执行 retire 时 请求与回复、门抄来的消息、syscall 回执、0 的事实行; run = 所属事件 seq; at = 收件箱偏移一次运行收笔: 起因 seq、发出的全部帧、异常
step	seq, actor, upto, out, err?, run?	

from 由介质盖章; 膜外来件署对应的门 (无对应门署 door)。物理地址 (数字 addr) 是 H 内位置、只增不复用、不遗传; 逻辑地址 channel/tag 写进 G 并遗传。

A.5 根门与收件箱

- 收件箱: 每 channel 一个 (Port.recv 侧), 来件写成 msg 投给该 channel 的接待员 (place 行带 in 标记者)。
- 根门: Space 级, 属于 R, 在一切 channel 之前存在, 不在 G 里。开着 ⇔ 所有账本无 msg 行 (由账本派生, 无隐藏状态)。开着时受理: channel.create, channel.add.actor (落账 by=_root)、以及第一条消息 (start, 正文携 G) ——它同时关门。关门后根门来件一律忽略。
- 生命周期边界只落根 channel: 出生是 start, 此后每次醒来/合法关闭是 _root → 接待员的 up / down 行; 各成员的重新实例化由该机器级事件与各自 place/retire 历史派生。

A.6 打包形态 P 与启动

P/omega.py runtime.py init.py	world 三文件 (= G.world 逐字节)
P/G.json	描述原样 (B 的产物)
P/h/<channel>.jsonl, h/_order	账本与介质引导索引 (运行时产生)
P/in/<box>.jsonl	收件箱: _root 与各 channel (运行时产生)
P/spawn/<name>/	子代的 P (繁殖时产生; 谱系即路径)
python init.py <P> [--serve]	起 R, 折叠已有账本, 驱动; 不读 G

B 证据索引

下列文件包含 §4 各表所引的原始账本行。账本随源码发布，快照位于 `evidence/runs/`；表内文件名均相对于该目录。凭据已抹 (*L* 源码中的 KEY 为 `sk-***`)。原始记录不改写：E1、E2 的账本生成于 `tag` 寻址之前，外部回执行用数字地址（如 `c2/5`）；E2 运行于生命周期语义收紧之前，重启事件逐 `channel` 注入。

实验	作者	文件	§4
E1-b 父代	deepseek-chat	task0-deepseek-c2.jsonl	§4.2
E1-b 子代	—	task0-deepseek-child-c2.jsonl	§4.2
E2-b d0 装机	deepseek-v4-pro	task1-d0-c2.jsonl； c3、c4 账本： task1-d0-c3.jsonl、 task1-d0-c4.jsonl	§4.3
E2-b/E2-c d1	同上	task1-d1-c4.jsonl (自组织、bug、唤醒)； task1-d1-c2.jsonl (修复)	§4.3
E2-b d2	—	task1-d2-c4.jsonl	§4.3
E3	桩（机制层）	t/test_c0.py, E3 驱动： 局部损伤、停机与再起、硬杀	§4.6
E4	桩（机制层）	t/test_c0.py, E4 驱动	§4.4
E5-b 父代	deepseek-v4-pro	restore-deepseek-c0.jsonl、 -c1.jsonl、 -c2.jsonl	§4.5
E5-b 子代	—	restore-deepseek-kid-c0.jsonl、 -kid-c1.jsonl	§4.5
E5-b 孙代	—	restore-deepseek-grand-c0.jsonl	§4.5

参考文献

- Gul Agha. Actors: A model of concurrent computation in distributed systems. Technical Report AI-TR-844, Massachusetts Institute of Technology, Artificial Intelligence Laboratory, June 1985. Actor behaviour triple in §2.1.3, p. 12; revised book edition published by MIT Press in 1986.
- Per Brinch Hansen. The nucleus of a multiprogramming system. *Communications of the ACM*, 13 (4):238–241, 1970. URL <https://doi.org/10.1145/362258.362278>.
- DeepSeek-AI. Deepseek harness: Everything is a plugin (developer preview). GitHub repository, 2026. URL <https://github.com/deepseek-ai/deepseek-harness>. MIT license; documentation at master commit 49a606bc5b59, retrieved 2026-09-02; earliest release tag 2026-08-17.
- Edsger W. Dijkstra. The structure of the “THE”-multiprogramming system. *Communications of the ACM*, 11(5):341–346, 1968. URL <https://doi.org/10.1145/363095.363143>.
- Russell Greiner and Douglas B. Lenat. A representation language language. In *Proceedings of the First Annual National Conference on Artificial Intelligence (AAAI-80)*, pages 165–169, Stanford, CA, August 1980. AAAI. URL <https://cdn.aaai.org/AAAI/1980/AAAI80-047.pdf>. The system is named RLL in this version; RLL-1 is the name used in Stanford HPP-80-9.
- Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, pages 235–245. Morgan Kaufmann Publishers Inc., 1973. URL <https://doi.org/10.5555/1624775.1624804>.

- Samuel Holt, Max Ruiz Luyten, and Mihaela van der Schaar. L2MAC: Large language model automatic computer for extensive code generation. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL <https://arxiv.org/abs/2310.02003>. v6 2025-06-27.
- Zae Myung Kim, Young-Jun Lee, Seungyeon Jwa, and Dongyeop Kang. Meta²: Recursive self-improvement through emergent depth, 2026. URL <https://arxiv.org/abs/2608.24735>. v1 2026-08-25.
- Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978. URL <https://doi.org/10.1145/359545.359563>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Hai Lin, Hoilam Pao, Shaoxiong Zhan, and Hai-Tao Zheng. Model-native computing architecture: Envisioning future system architecture through the lens of computer architecture, 2026a. URL <https://arxiv.org/abs/2606.00288>. v1 2026-05-29, v3 2026-06-24.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shiha Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026b. URL <https://arxiv.org/abs/2604.25850>.
- Ryan Lopopolo. Harness engineering: Leveraging Codex in an agent-first world. OpenAI, February 2026. URL <https://openai.com/index/harness-engineering/>. Byline Ryan Lopopolo, Member of the Technical Staff, dated February 11, 2026; verified via Wayback Machine snapshot 20260830095239 (direct access returned 403).
- Humberto R. Maturana and Francisco J. Varela. *Autopoiesis and Cognition: The Realization of the Living*, volume 42 of *Boston Studies in the Philosophy of Science*. D. Reidel, Dordrecht, 1980. ISBN 90-277-1015-5. URL <https://doi.org/10.1007/978-94-009-8947-4>. Definition of autopoietic organization, pp. 78–79; wording verified against the scanned book.
- Barry McMullin. John von neumann and the evolutionary growth of complexity: Looking backward, looking forward. *Artificial Life*, 6(4):347–361, October 2000. URL <https://doi.org/10.1162/106454600300103674>.
- Yapeng Mi, Zhi Gao, Xiaojian Ma, and Qing Li. Building LLM agents by incorporating insights from computer systems, 2025. URL <https://arxiv.org/abs/2504.04485>. v1 2025-04-06.
- Eric Nivel, Kristinn R. Thórisson, Bas R. Steunebrink, Haris Dindo, Giovanni Pezzulo, Manuel Rodriguez, Carlos Hernandez, Dimitri Ognibene, Jürgen Schmidhuber, Ricardo Sanz, Helgi P. Helgason, Antonio Chella, and Gudberg K. Jonsson. Bounded recursive self-improvement. Technical Report RUTR-SCS13006, Reykjavik University, 2013. URL <https://arxiv.org/abs/1312.6764>. v1 2013-12-24.
- Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981. URL <https://doi.org/10.1016/j.jlap.2004.05.001>. Reprinted in *Journal of Logic and Algebraic Programming* 60–61 (2004), 17–139.
- Prithvi Rajasekaran. Harness design for long-running application development. Anthropic Engineering, March 2026. URL <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. *The Bell System Technical Journal*, 57(6, part 2):1905–1929, 1978. URL <https://doi.org/10.1002/j.1538-7305.1978.tb02136.x>. Revised version of the CACM 17(7), 1974 paper; quotations follow this version.
- Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03, IDSIA, 2003. URL <https://arxiv.org/abs/cs/0309048>. v5 2006-12-17.
- Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. URL <https://arxiv.org/abs/2608.25512>.
- Brian Cantwell Smith. Reflection and semantics in LISP. In *Proceedings of the 11th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '84)*, pages 23–35, Salt Lake City, Utah, January 1984. ACM. URL <https://doi.org/10.1145/800017.800513>.

- John von Neumann. First draft of a report on the EDVAC. Technical report, Moore School of Electrical Engineering, University of Pennsylvania, June 1945. URL <https://doi.org/10.5479/sil.538961.39088011475779>. Dated June 30, 1945; reprinted by M. D. Godfrey in *IEEE Annals of the History of Computing* 15(4): 27–75, 1993.
- John von Neumann. The general and logical theory of automata. In Lloyd A. Jeffress, editor, *Cerebral Mechanisms in Behavior: The Hixon Symposium*, pages 1–41. John Wiley & Sons, New York, 1951. Lecture delivered September 20, 1948.
- John von Neumann. *Theory of Self-Reproducing Automata*. University of Illinois Press, Urbana, 1966. Edited and completed by Arthur W. Burks.
- Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026. URL <https://arxiv.org/abs/2607.12227>.
- Lilian Weng. Harness engineering for self-improvement. Lil’Log, July 2026. URL <https://lilianweng.github.io/posts/2026-07-04-harness/>.
- Simon Yu, Derek Chong, Ananjan Nandi, Dilara Soylu, Jiuding Sun, Christopher D. Manning, and Weiyang Shi. Shepherd: Enabling programmable meta-agents via reversible agentic execution traces, 2026. URL <https://arxiv.org/abs/2605.10913>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling (COLM 2024)*, 2024. URL <https://arxiv.org/abs/2310.02304>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025. URL <https://arxiv.org/abs/2505.22954>.
- Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents, 2026a. URL <https://arxiv.org/abs/2603.19461>.
- Jiang Zhang, Bing Yuan, and Qian Zhang. Self-reference in large language models: The introspection threshold for recursive self-improvement, 2026b. URL <https://arxiv.org/abs/2607.04277>. v1 2026-07-05, 21 pages.
- Yingqi Zhang. Agent libos: A runtime substrate for capability-controlled self-evolving llm agents, 2026. URL <https://arxiv.org/abs/2606.03895>.